

Measuring performance

Before we talk about how to optimize performance we need ways to effectively measure it. But even before we can measure it we need to know what exactly we want to measure.

Here are some of the most important performance measures of an API/backend service:

- Throughput: the number of requests the system can handle without going down.
- Load time: the amount of time it takes for an HTTP request to respond.
- Size: the total size of the HTTP response.
- Server uptime: the duration of time the server is up and running usually expressed as a percentage.
- CPU usage: the amount of CPU your system needs to run. It is usually expressed as load average which I'm gonna explain later.
- Memory usage: the amount of memory your system uses.

In this book, we're going to talk about backend and APIs but of course, there are some frontend-related metrics as well:

- Load time: the amount of time it takes for the full page to load.
- First byte: the time taken to start loading the data of your web application after a user requests it.
- Time to interactive: this measures how long it takes a page to become fully interactive, i.e., the time when the layout has stabilized, key web fonts are visible, and the main thread is available enough to handle user input.
- Page size: the total size of the web page, that includes all of its resources (HTML, CSS, JavaScript, images, etc).
- Number of requests: the number of individual requests made to the server to fully load the web page.

These things are "black box measures" or "external measures." Take load time for an example and say the `GET api/products` endpoint took `912s` to load which is slow. Measuring the load time tells you that your system is slow, but it doesn't tell you why. To find out the cause we need to dig deeper into the black box. We need to debug things such as:

- Number of database queries
- The execution time of database queries
- Which function takes a long time to finish its job?
- Which function uses more memory than it's supposed to?
- What parts of the system can be async?
- and so on

Measuring a system from the outside (for example load time of an API endpoint) is always easier than measuring the internal parts. This is why we start with the external measures first.

ab

The easiest tool to test your project's performance is `ab` or Apache Benchmark. It's a command line tool that sends requests to a given URL and then shows you the results.

You can use it like this:

```
ab -n 100 -c 10 -H "Authorization: Bearer 1|A7dIitFpmzsDATwEqmBQzDtfdHkcWCTfGCvO197u"
http://127.0.0.1:8000/api/transactions
```

This sends `100` requests to `http://127.0.0.1:8000/api/transactions` with a concurrency level of `10`. This means that 10 of those requests are concurrent. They are sent at the same time. These concurrent requests try to imitate multiple users using your API at the same time. It will send 10 requests at a time until it reaches 100.

Unfortunately, in `ab` we cannot specify the ramp-up time. This is used to define the total time in which the tool sends the requests to your app. For example, "I want to send 100 requests in 10 seconds." You cannot do that with `ab`. It will always send requests when it can. So if the first batch of the concurrent requests (which is 10 requests in this example) is finished in 3 seconds then it sends the next batch and so on. Other than that, it's the perfect tool to quickly check the throughput of your application.

And now, let's interpret the results:

```
Concurrency Level:      10
Time taken for tests:    2.114 seconds
Complete requests:      100
Failed requests:        0
Total transferred:      1636000 bytes
HTML transferred:      1610100 bytes
Requests per second:    47.31 [#/sec] (mean)
Time per request:       211.363 [ms] (mean)
Time per request:       21.136 [ms] (mean, across all concurrent requests)
Transfer rate:          755.88 [Kbytes/sec] received
```

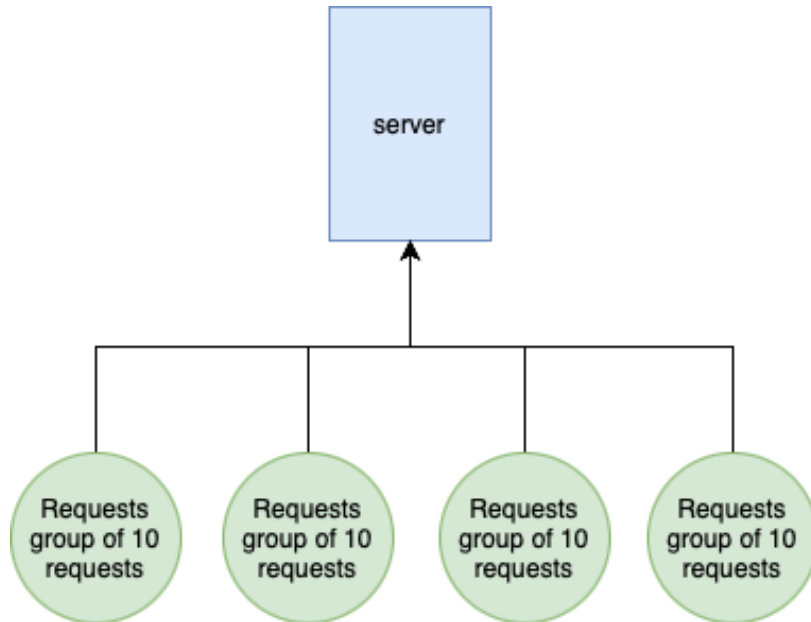
As you can see, it sent a total of 100 requests with a concurrency level of 10. The whole test took 2114ms or 2.114 seconds. If we divide 100 by 2.114 seconds the result is 47.31. This is the **throughput** of the server. It can handle 47 requests per second.

These tests were made on my MacBook with a pretty simple API endpoint so they are quite good. Later, I'm going to destroy some \$6 DigitalOcean servers and I'll also try a \$1300 one with 48 CPUs.

The next two numbers were quite hard for me to understand at first. They are:

- Time per request: 211.363 [ms] (mean)
- Time per request: 21.136 [ms] (mean, across all concurrent requests)

When you run `ab -n 100 -c 10` `ab` creates 10 request "groups" that contain 10 requests each:



In this case `Time per request: 21.136 [ms] (mean, across all concurrent requests)` means that **1 request took 21ms on average**. This is the important number.

The other `Time per request: 211.363 [ms] (mean)` refers to a request group. Which contains 10 requests. You can clearly see the correlation between these numbers:

- Time taken for tests: 2114 ms
 - All 100 requests took a total of 2114 ms
- Time per request: 21.136 [ms] (mean, across all concurrent requests)
 - $2114 \text{ ms} / 100 \text{ requests} = 21.14 \text{ ms per request}$
 - One request took 21.14 ms on average
- Time per request: 211.363 [ms] (mean)
 - $2114 \text{ ms} / 100 \text{ requests} * 10 = 211.4 \text{ ms}$
 - One request group of 10 requests took 211 ms on average

So if you use concurrency the last number doesn't really make sense. It was really confusing for me at first, so I hope I gave you a better explanation.

`ab` is a fantastic tool because it is:

- Easy to install
- Easy to use
- You can load test your app in minutes and get quick results

But of course, it has lots of limitations.

jmeter

The next tool to load test your application is `jmeter`. It has more advanced features than `ab` including:

- Defining ramp-up period
- Building a "pipeline" of HTTP requests simulating complex user interactions
- Adding assertions (response validation) to your HTTP tests
- Better overview of your app's performance
- Test results visualization using charts, graphs, tree view, etc.
- Other useful testing features such as XPath, regular expression, JSON, script variables, and response parsing, that help us to build more exact and effective tests.
- GUI

A quick note. If you're having trouble starting jmeter try this command with the `UserG1GC` argument: `java -XX:+UnlockExperimentalVMOptions -XX:+UseG1GC -jar ApacheJMeter.jar`. You can also use this alias: `alias jmeter='JVM_ARGS="-XX:+UnlockExperimentalVMOptions -XX:+UseG1GC" /path/to/jmeter/bin/jmeter'`

To start load testing with jmeter you need to create a new `Thread group` that has the following options:

- `Number of threads` refers to the number of users or put it simply, the number of requests.
- `Ramp-up period` defines how much time should jmeter take to start the requests. If 10 threads are used, and the ramp-up period is 100 seconds, then jmeter will take 100 seconds to get all 10 threads up and running. Each thread will start 10 (100/10) seconds after the previous thread was begun.
- And then we have `Loop count`. By default, it's 1 meaning that jmeter runs your HTTP tests once. If you set it to 100 it repeats all of the tests 100 times.

Note: you can find the example test plan in the source code `1-measuring-performance/simple-jmeter-test-plan.jmx`

Inside the `Thread group` we need to add an `HTTP Sampler` which can be found inside the `Sampler` category. An HTTP request is pretty straightforward. You just need to configure the base URL, the endpoint, and query or POST body parameters if you have any.

In order to measure the results we need to add `Listeneres` as well. These are the components that can display the results in various formats. Two of the most crucial listeners are the `Summary Report` and the `View Results Tree`. Add them to the Thread group.

Summary report looks like this:

Summary Report

Name:

Summary Report

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only:

☐ Errors

☐ Successes

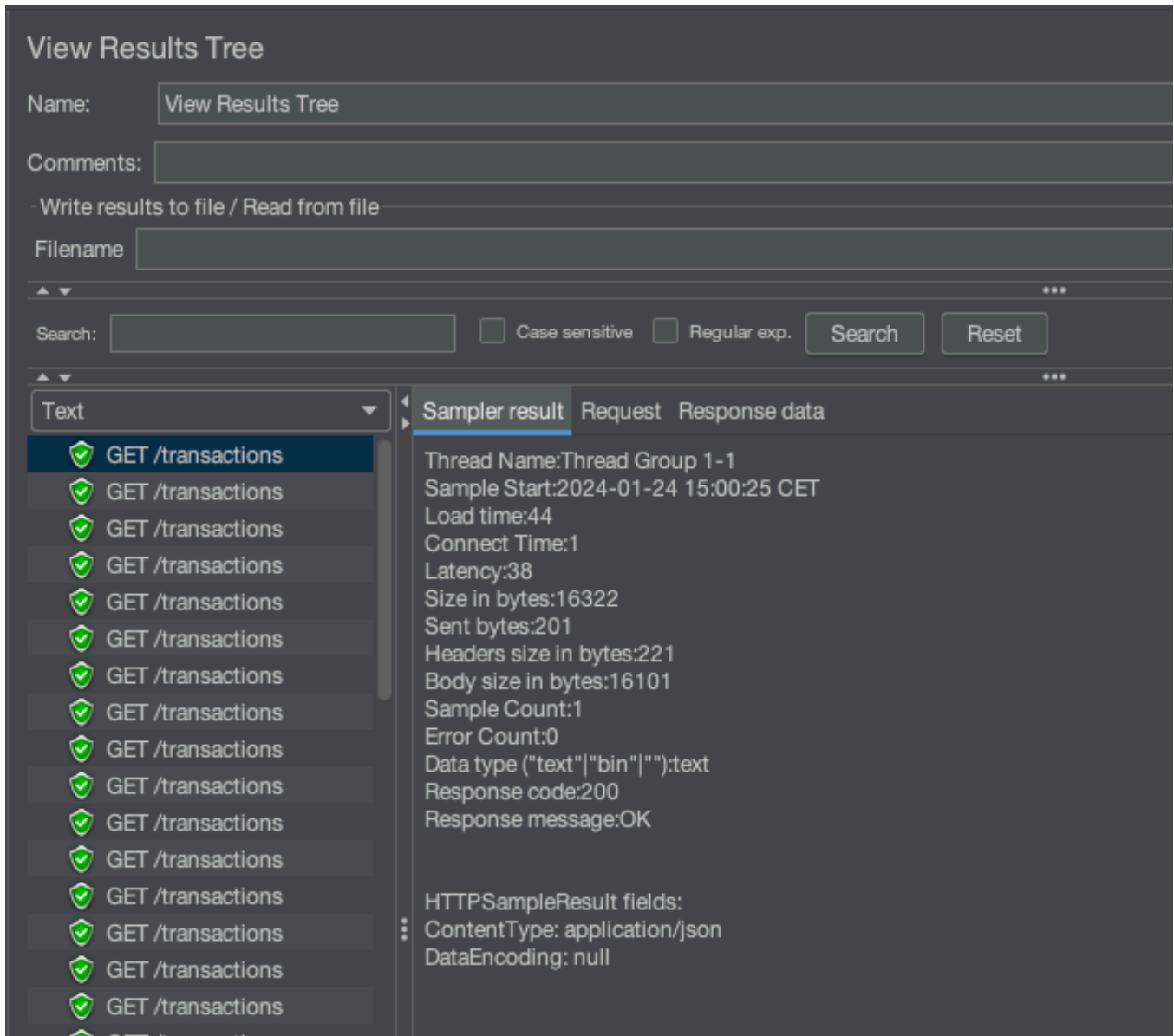
Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
GET /transactio...	100	55	27	104	27.31	0.00%	29.6/min	7.87	0.10	16322.0
TOTAL	100	55	27	104	27.31	0.00%	29.6/min	7.87	0.10	16322.0

- Average: the average response time in ms
- Median: the median (50th percentile) response time
- Max: the slowest response time
- Std. Dev: it's the standard deviation during the test. In general, standard deviation shows you the amount of variation or dispersion of a set of values. In performance testing, the standard deviation indicates how much individual sample times deviate from the average response time. In this example, 27 indicates that the individual sample times in the dataset are, on average, 27 units away from the mean of the dataset. This means that there is a relatively high level of variability or dispersion in the sample times.
- Error%: the percentage of requests that resulted in an error (non-2xx)
- Throughput: the number of requests per minute that can be served by the server
- Received KB/Sec: the amount of data received per second during the test.

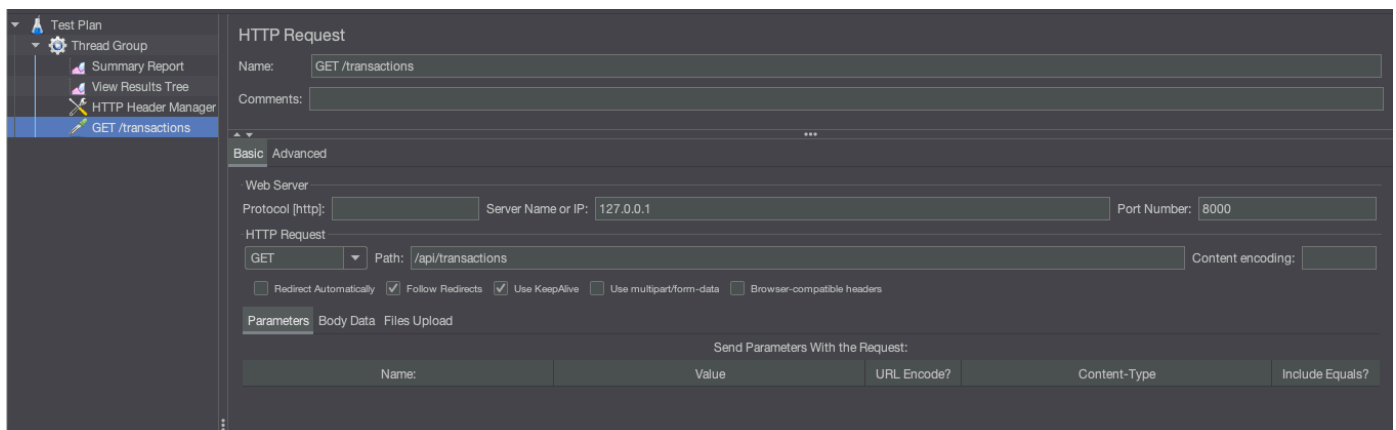
So the summary report gives you a quick overview of the overall results of your tests.

`View Results Tree` on the other hand enables you to check out individual requests which can be helpful if you have 5xx responses. It looks like this:



The last thing you probably need is to send the `Authorization` header in your requests. In jmeter there's a dedicated component to set header values. It's called `HTTP Header Manager` and can be found in the `Managers` category. The setup is really easy. You just need to add the header's name and value.

So a simple but working test plan looks like this:



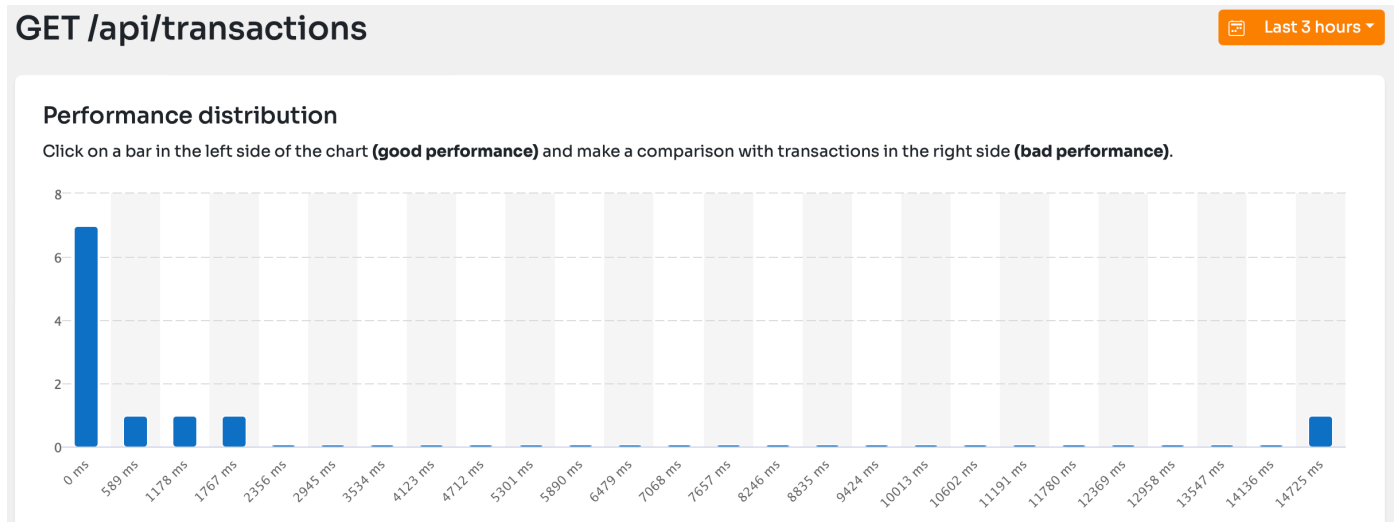
Note: you can find this example test plan in the source code `1-measuring-performance/simple-jmeter-test-plan.jmx`

Inspector

`ab` and `jmeter` are great if you want to understand the throughput and overall responsiveness of your application. Let's say you found out that the `GET /api/transactions` endpoint is slow. Now what? You open the project and go to the Controller trying to find the slow part. You might add some `dd` or `time()` and so on. Fortunately, there's a better way.

[Inspector.dev](#) allows you to visualize the internals of your applications.

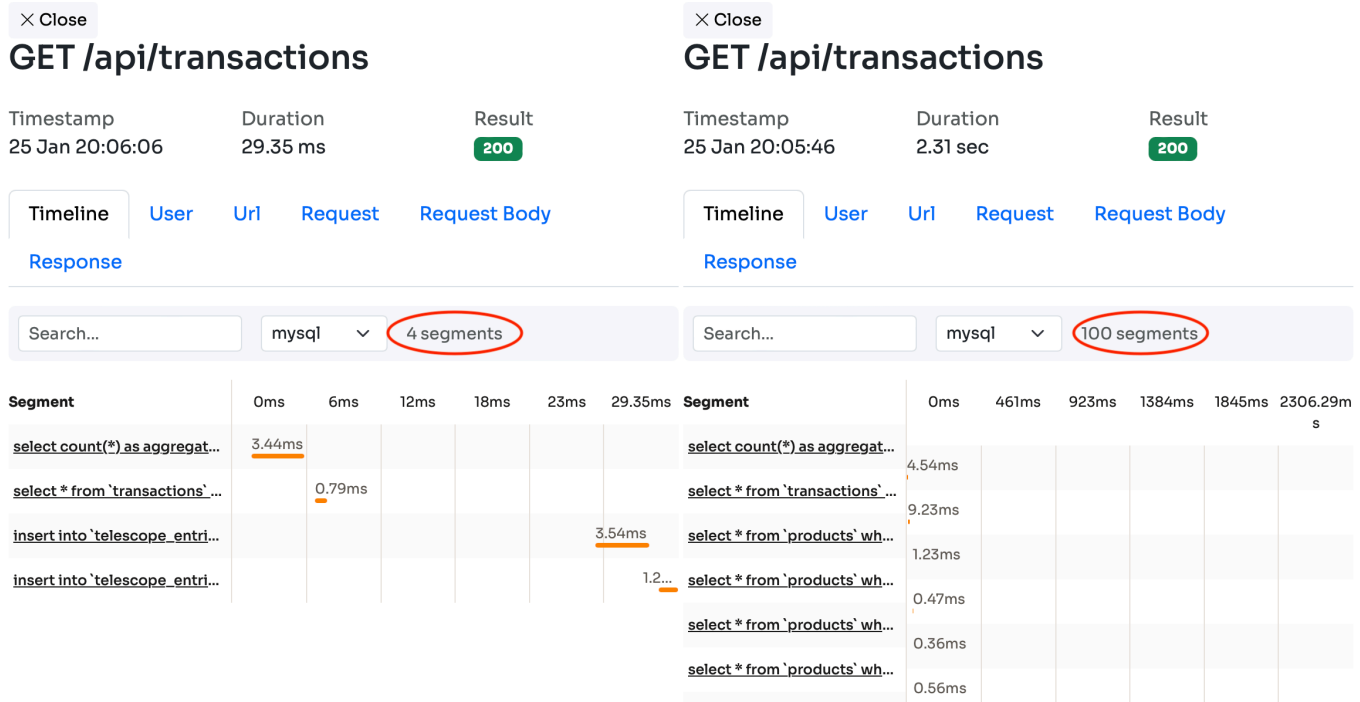
For example, here's a time distribution of the different occurrences of the `GET /api/transactions` request:



I sent the request 11 times:

- 7 times it took only 20-50ms. You can see these occurrences on the left side on the `0ms` mark.
- 3 times it took something between 500ms and 2500ms. These are the 3 smaller bars.
- And then one time it took almost 15 seconds. This is the lonely bar on the right side.

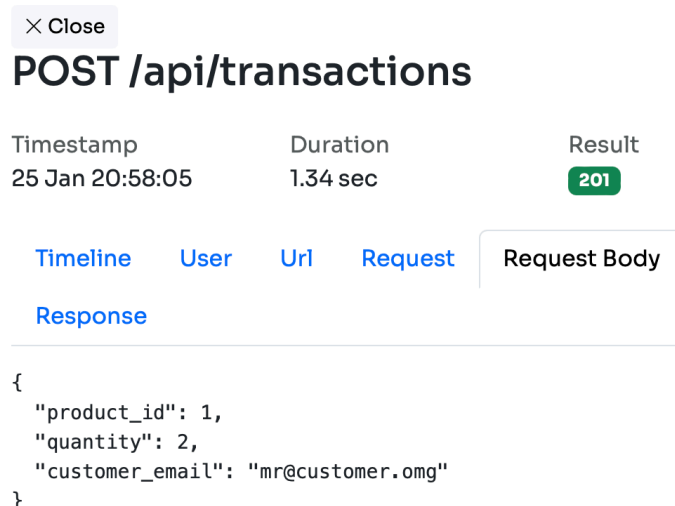
If I click on these bars I can quickly what's the difference between a 29ms and a 2300ms request:



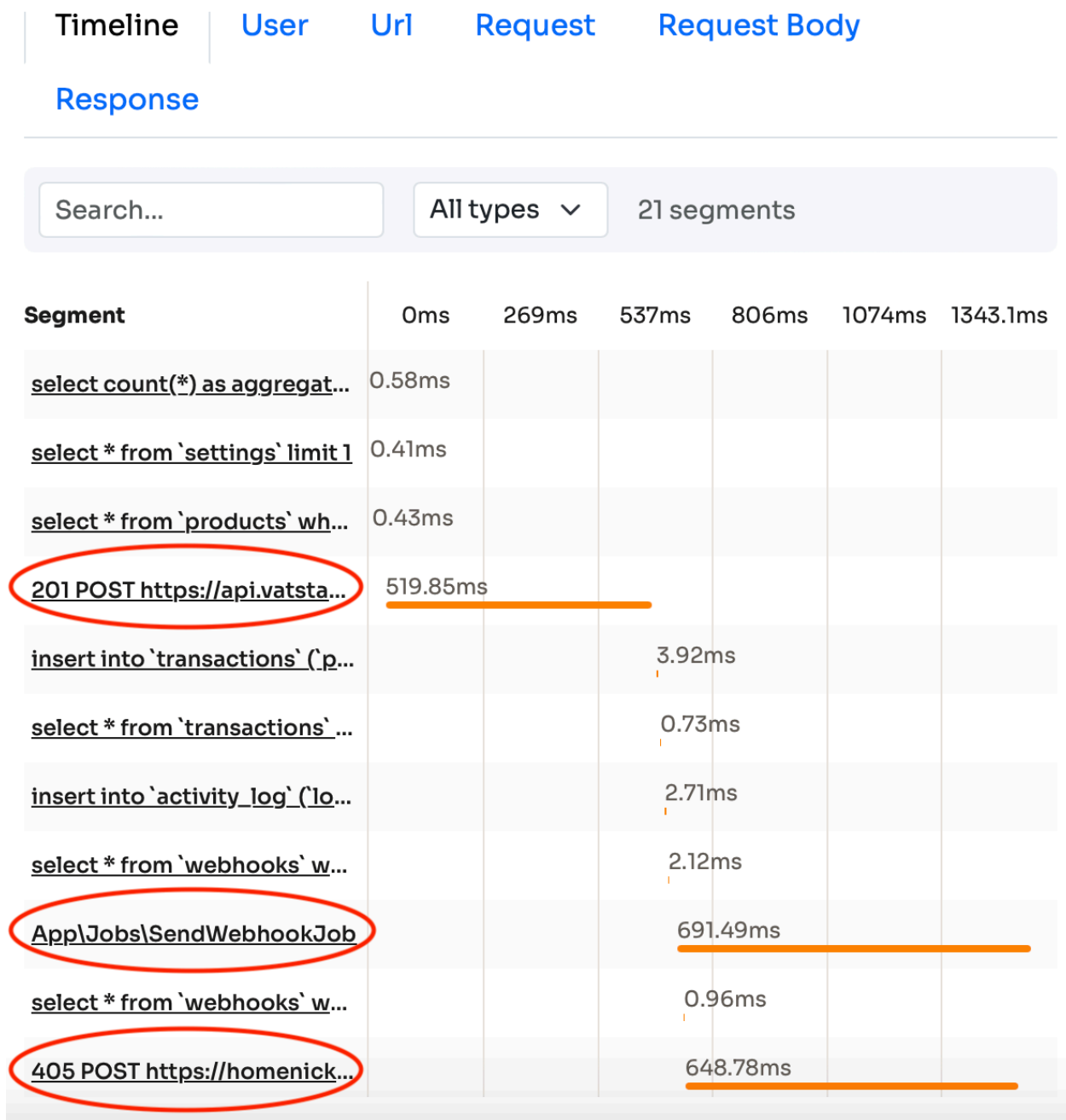
In the left panel, you can see that only 4 MySQL queries were executed and the request took 29ms to complete. On the right side, however, there were 100 queries executed and it took 2.31 seconds. You can see the individual queries as well. On the right side, there are these extra `select * from products` queries that you cannot see on the left side.

In the `user` menu, you can always check out the ID of the user that sent the request. It's a great feature since user settings and different user data can cause differences in performance.

If it's a `POST` request you can see the body in the `Request Body` menu:

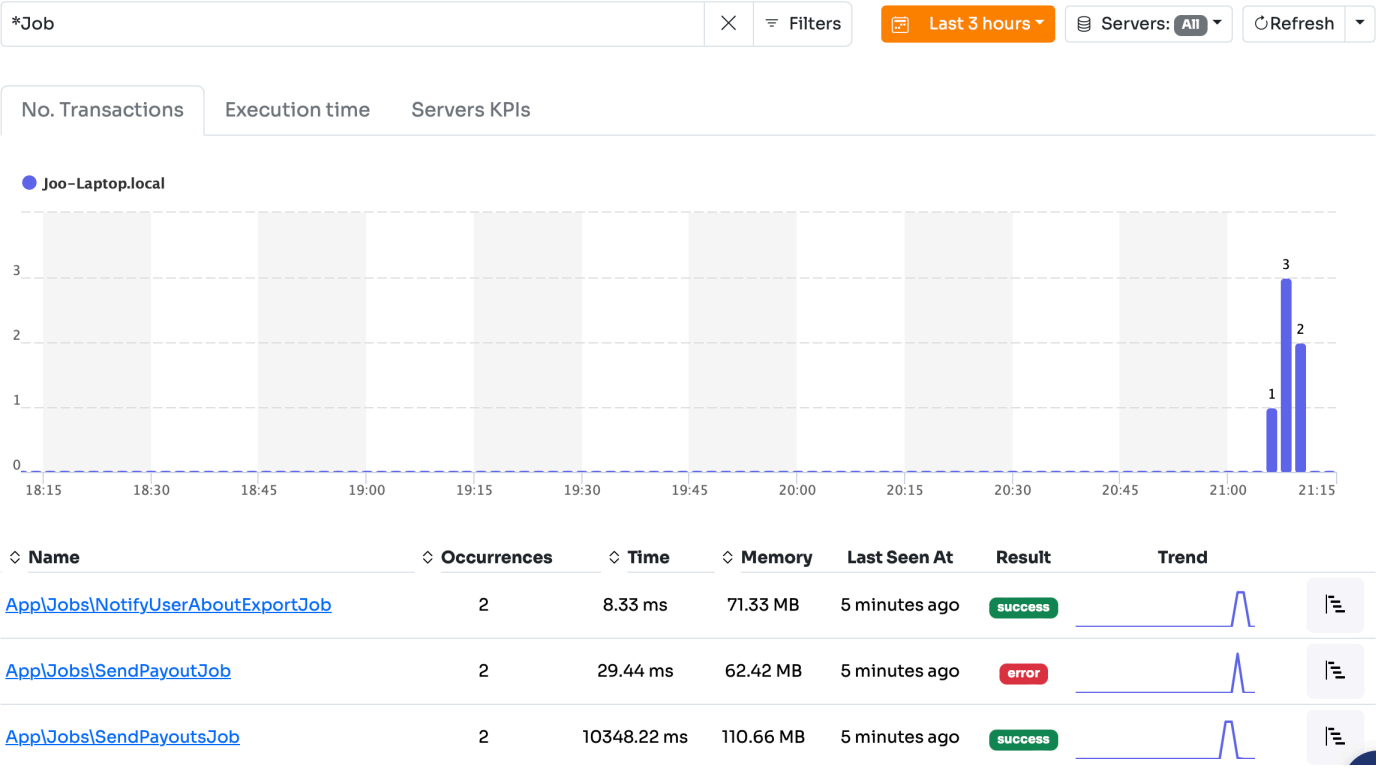


Another great feature of Inspector is that it also shows outgoing HTTP requests and dispatched jobs in the timeline:



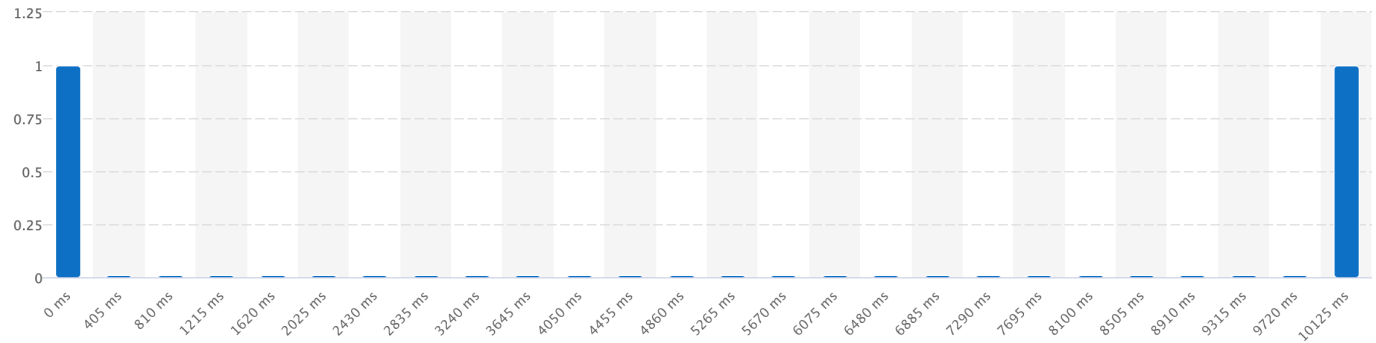
In my example application, the `POST /api/transactions` endpoint communicates with other APIs and also dispatched a job. These are the highlighted rows on the image.

The great thing about Inspector is that it integrates with Laravel so it can detect things like your queue jobs:



You can dig into the details of jobs just like HTTP requests:

Performance distribution
Click on a bar in the left side of the chart (**good performance**) and make a comparison with transactions in the right side (**bad performance**).



×

Close

App\Jobs\SendPayoutsJob

Timestamp

25 Jan 21:09:59

Duration

232.88 ms

Result

success

Timeline

Payload

×

Close

App\Jobs\SendPayoutsJob

Timestamp

25 Jan 21:06:51

Duration

10.35 sec

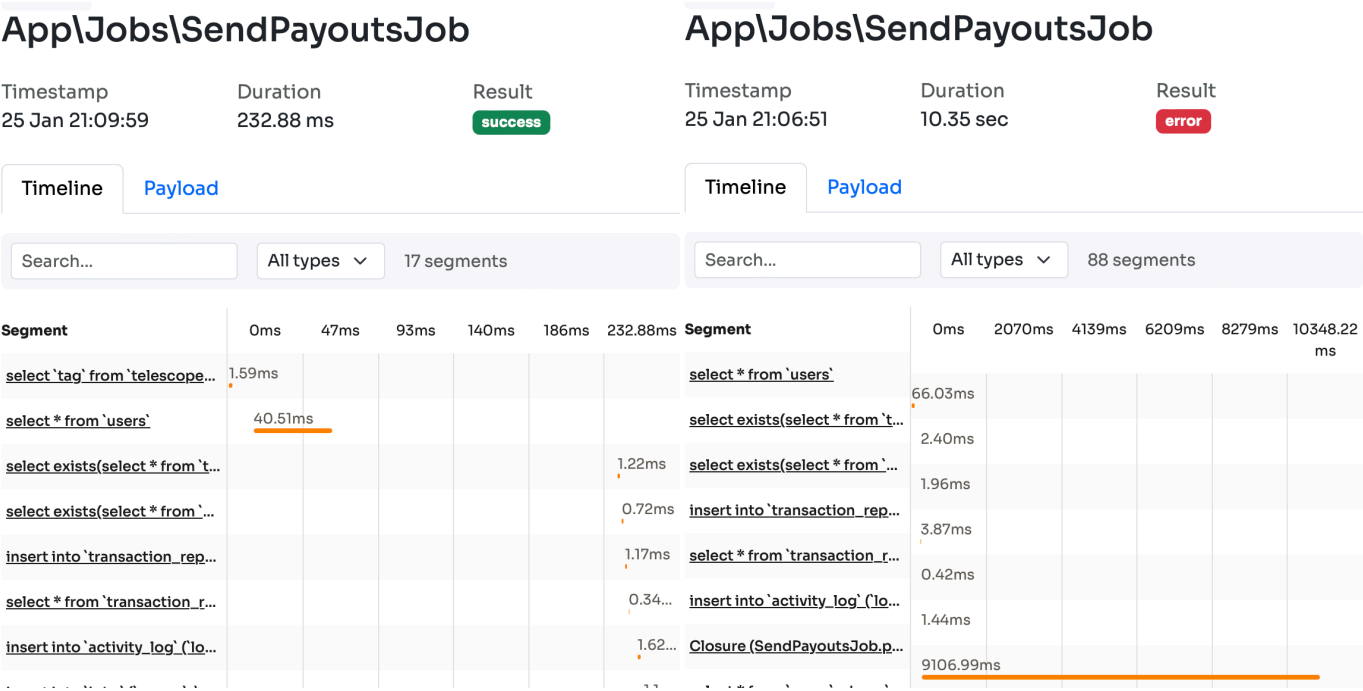
Result

error

Timeline

Payload

You have the same comparasion view with all the database queries, HTTP requests, or other dispatched jobs:



The best thing about Inspector? This is the whole installation process:

```
composer require inspector-apm/inspector-laravel
```

We'll use Inspector later in the book, it was just a quick introduction. Check out their [docs](#) here.

Telescope

Even though, Inspector is awesome, it's a paid 3rd party tool so I understand not everyone wants to use it.

One of the easiest tools you can use to monitor your app is [Laravel Telescope](#).

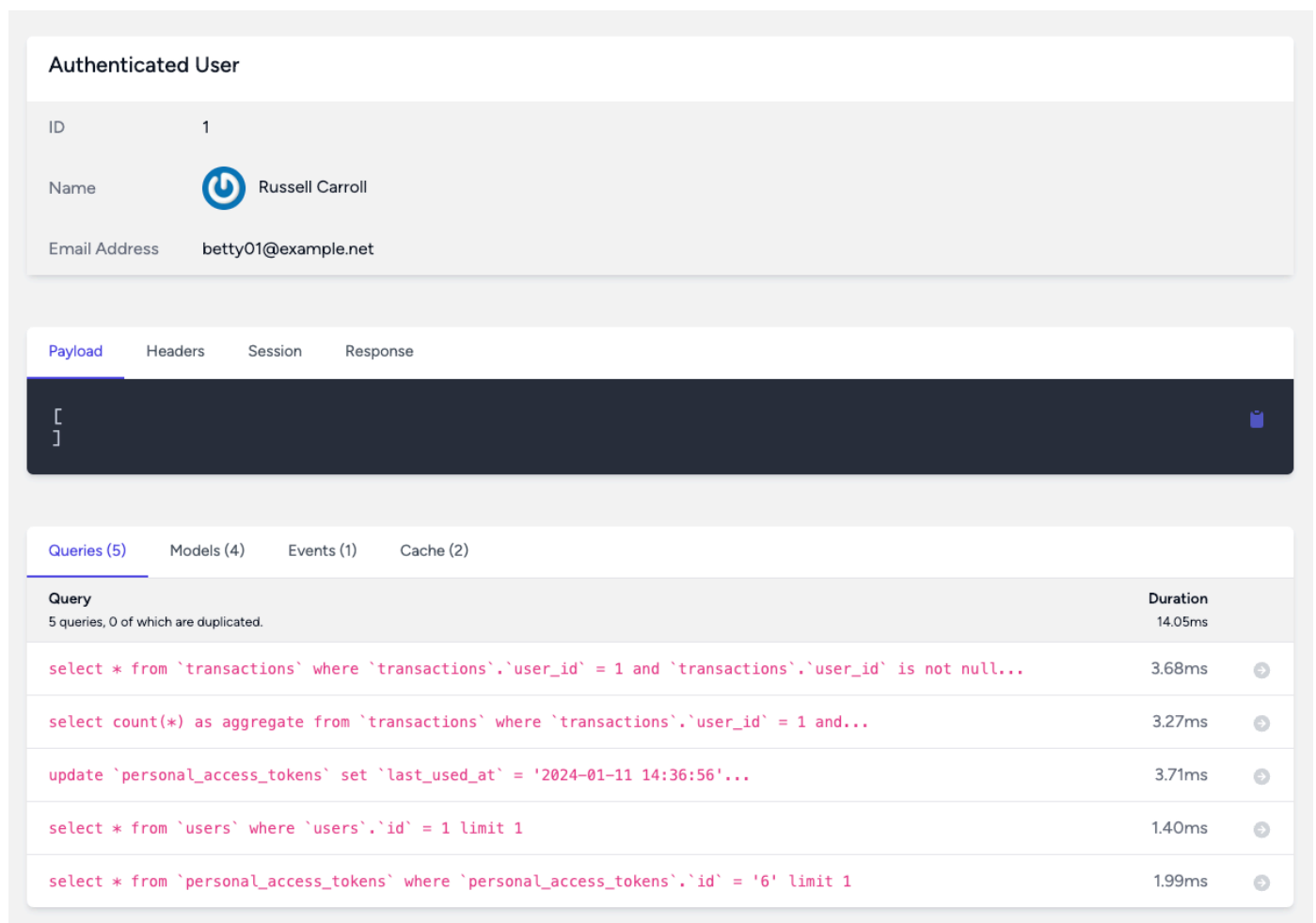
After you've installed the package you can access the dashboard at `localhost:8000/telescope`. If you send some requests you'll see something like that:



The screenshot shows the Laravel Telescope dashboard for a user named 'Piddle'. The left sidebar contains navigation links for Requests, Commands, Schedule, Jobs, Batches, Cache, Dumps, and Events. The main area displays a table of requests with columns for Verb, Path, Status, Duration, and Happened. The table lists several GET requests to various API endpoints, including /api/webhooks, /api/products, and /api/transactions. The status of the requests is indicated by colored dots (green for 200, yellow for 404).

Verb	Path	Status	Duration	Happened
GET	/api/webhooks	200	76ms	4s ago
GET	/api/products	404	27ms	21s ago
GET	/api/transactions/9b0bc33b-be23-4d98-a093-7a62e...	200	32ms	29s ago
GET	/api/transactions	200	66ms	31s ago
GET	/api/transactions	200	103ms	1:14m ago

It gives you a great overview of your requests and their duration. What's even better, if you click on a specific request you can see all the database queries that were executed:



The screenshot shows the details of a specific request in the Laravel Telescope dashboard. The top section displays the 'Authenticated User' information, including the ID (1), Name (Russell Carroll), and Email Address (betty01@example.net). Below this, the 'Payload' tab is selected, showing an empty array. The bottom section displays the 'Queries' tab, showing a list of database queries executed during the request. The queries are listed with their duration and a link to view the full query details.

Query	Duration
5 queries, 0 of which are duplicated.	14.05ms
select * from `transactions` where `transactions`.`user_id` = 1 and `transactions`.`user_id` is not null...	3.68ms
select count(*) as aggregate from `transactions` where `transactions`.`user_id` = 1 and...	3.27ms
update `personal_access_tokens` set `last_used_at` = '2024-01-11 14:36:56'...	3.71ms
select * from `users` where `users`.`id` = 1 limit 1	1.40ms
select * from `personal_access_tokens` where `personal_access_tokens`.`id` = '6' limit 1	1.99ms

If you click on an entry you can see the whole query and the request's details:

Query Details

Time	January 11th 2024, 9:28:07 PM (7s ago)
Hostname	Joo-MacBook-Air.local
Connection	mysql
Location	/Users/joomartin/code/paddle/routes/api.php:60
Duration	45.06ms
Request	View Request

Query

```
select
  `uuid`,
  `product_data`,
  `quantity`,
  `revenue`,
  `fee_amount`,
  `tax_amount`,
  `balance_earnings`,
  `customer_email`,
  `created_at`
from
  `transactions`
where
  `user_id` = 1
order by
  `created_at` asc
limit
  250 offset 0
```

Telescope can also monitor lots of other things such as:

- Commands
- Jobs
- Cache
- Events
- Exceptions
- Logs
- Mails
- ...and so on

For example, here are some queue jobs after a laravel-excel export:

Laravel Telescope - Piddle

Requests
Commands
Schedule
Jobs
Batches
Cache
Dumps
Events
Exceptions
Gates
HTTP Client
Logs
Mail
Models
Notifications
Queries
Redis

Jobs

Search Tag

Job	Status	Happened
App\Notifications\ExportReadyNotification Connection: database Queue: default	processed	1:06m ago
App\Jobs\NotifyUserAboutExportJob Connection: database Queue: default	processed	1:06m ago
Maatwebsite\Excel\Jobs\StoreQueuedExport Connection: database Queue: default	processed	1:06m ago
Maatwebsite\Excel\Jobs\CloseSheet Connection: database Queue: default	processed	1:08m ago
Maatwebsite\Excel\Jobs\AppendQueryToSheet Connection: database Queue: default	processed	1:09m ago
Maatwebsite\Excel\Jobs\AppendQueryToSheet Connection: database Queue: default	processed	1:10m ago
Maatwebsite\Excel\Jobs\AppendQueryToSheet Connection: database Queue: default	processed	1:12m ago
Maatwebsite\Excel\Jobs\AppendQueryToSheet Connection: database Queue: default	processed	1:13m ago
Maatwebsite\Excel\Jobs\AppendQueryToSheet Connection: database Queue: default	processed	1:14m ago

Telescope is a great tool and it's a must-have if you want to monitor and improve your app's performance. If you want to use only free and simple tools go with `ab` and Telescope. `ab` tells you what part of the app is slow. Telescope tells you why it's slow.

OpenTelemetry

Both Inspector and Telescope track everything by default, which is a great thing. However, sometimes you might want to control what's being tracked and what is not.

To do that, the best option in my opinion is OpenTelemetry. OpenTelemetry is an Observability framework and toolkit designed to create and manage telemetry data such as traces, metrics, and logs. It's independent of languages, tools, or vendors. It offers a standardized specification and protocol that can be implemented in any language.

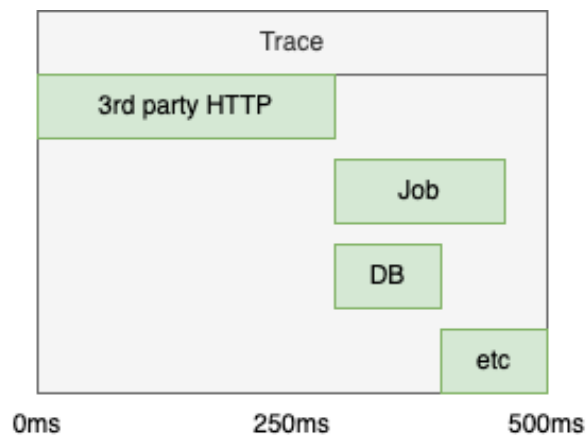
There are two important OpenTelemetry terms, they are:

- Traces
- Spans

A **trace** is a set of events. It's usually a complete HTTP request and contains everything that happens inside. Imagine if your API endpoint sends an HTTP request to a 3rd party, dispatches a job, sends a Notification, and runs 3 database queries. All of this is one trace. Every trace has a unique ID.

A **span** is an operation inside a trace. The HTTP request to the 3rd party can be a span. The dispatched job can be another one. The notification can be the third one. Finally, you can put the 3 queries inside another span. Each span has a unique ID and they contain the trace ID. It's a parent-child relationship.

We can visualize it like this:



So it's similar to Inspector, however, it requires manual instrumentation. Instrumentation means you need to start and stop the traces manually, and then you need to add spans as you like to. So it requires more work but you can customize it as you wish.

OpenTelemetry offers a [PHP SDK](#). However, the bare bone framework is a bit complex to be honest, so I'll use a simple but awesome Spatie package to simplify the whole process. It's called [laravel-open-telemetry](#).

The installation steps are easy:

```
composer require spatie/laravel-open-telemetry
php artisan open-telemetry:install
```

To start using it we need to manually add the spans:

```
Measure::start('Communicating with 3rd party');

Http::get('...');

Measure::stop('Communicating to 3rd party');
```

The `start` method starts a new **span**. Behind the scenes, a unique trace ID will be generated at the start of **every request**. When you call `Measure::start()` a span will be started that will get that trace ID injected. So we only worry about spans. Traces are handled by the package.

But what happens with these traces and spans? How can I view them? Great question!

The collected data needs to be stored somewhere and needs a frontend. We need to run some kind of store and connect it to the Spatie package. There are multiple tracing system that handles OpenTelemetry data. For example, [ZipKin](#) or [Jaeger](#). I'm going to use ZipKin since it's the most simple to set up locally. All we need to do is this:

```
docker run -p 9411:9411 openzipkin/zipkin
```

Now the the Zipkin UI is available at <http://localhost:9411>.

In the `open-telemetry.php` config file, we can configure the driver:

```
'drivers' => [
    Spatie\OpenTelemetry\Drivers\HttpDriver::class => [
        'url' => 'http://localhost:9411/api/v2/spans',
    ],
],
```

Now Spatie will send the collected metrics to `localhost:9411` where ZipKin listens.

Let's see an example of how we can add these spans. When you purchased this book (thank you very much!) you interacted with Paddle even if you didn't realize it. It's merchant of record meaning you paid for them and they will send me the money once a month. This way, I worry about only one invoice a month. They also handle VAT ramifications.

So imagine an endpoint when we can buy a product, let's call it: `POST /api/transactions` the requests look like this:


```
namespace App\Http\Requests;

class StoreTransactionRequest extends FormRequest
{
    public function rules(): array
    {
        return [
            'product_id' => ['required', 'exists:products,id'],
            'quantity' => ['required', 'numeric', 'min:1'],
            'customer_email' => ['required', 'email'],
        ];
    }
}
```

It's a simplified example, of course. When someone buys a product we need to do a number of things:

- Calculating the VAT based on the customer's IP address
- Inserting the actual DB record
- Triggering a webhook means we call some user-defined URLs with the transaction's data
- Moving money via Stripe. We'll skip that part now.

Calculating the VAT involves talking to 3rd party services (for example VatStack). The `transactions` table can be huge in an application like this so it's a good idea to place a span that contains this one query specifically.

We can add spans like these:

```
public function store(StoreTransactionRequest $request, VatService $vatService, Setting $setting)
{
    Measure::start('Create transaction');

    $product = $request->product();

    /** @var Money $total */
    $total = $product->price->multiply($request->quantity());

    Measure::start('Calculate VAT');

    $vat = $vatService->calculateVat($total, $request->ip());

    Measure::stop('Calculate VAT');

    $feeRate = $setting->fee_rate;

    $feeAmount = $total->multiply((string) $feeRate->value);

    Measure::start('Insert transaction');

    $transaction = Transaction::create([
```

```

'product_id' => $product->id,
'quantity' => $request->quantity(),
'product_data' => $product->toArray(),
'user_id' => $product->user_id,
'stripe_id' => Str::uuid(),
'revenue' => $total,
'fee_rate' => $feeRate,
'fee_amount' => $feeAmount,
'tax_rate' => $vat->rate,
'tax_amount' => $vat->amount,
'balance_earnings' => $total->subtract($vat->amount)->subtract($feeAmount),
'customer_email' => $request->customerEmail(),
]);

Measure::stop('Insert transaction');

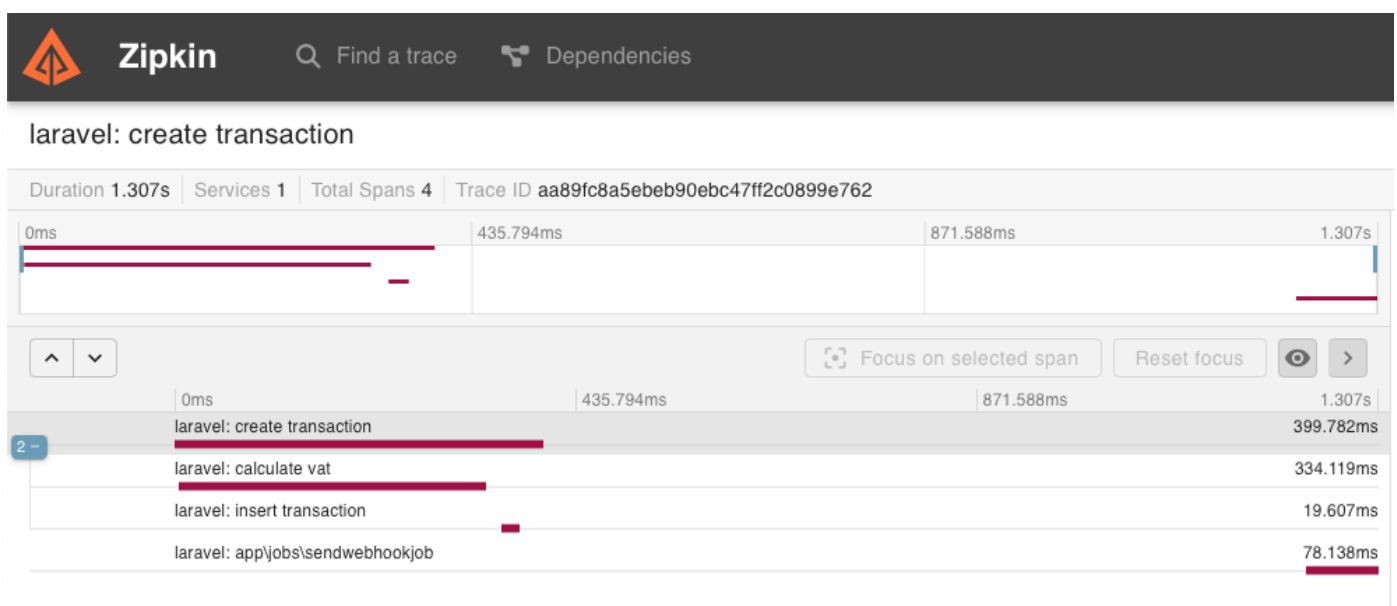
try {
    if ($webhook = Webhook::transactionCreated($request->user())) {
        SendWebhookJob::dispatch($webhook, 'transaction_created', [
            'data' => $transaction,
        ]);
    }
} catch (Throwable) {}

Measure::stop('Create transaction');

return response([
    'data' => TransactionResource::make($transaction)
], Response::HTTP_CREATED);
}

```

Here's the result in ZipKin:



The trace is called `laravel: create transaction` where `laravel` comes from the default config of the package and `create transaction` comes from the first span.

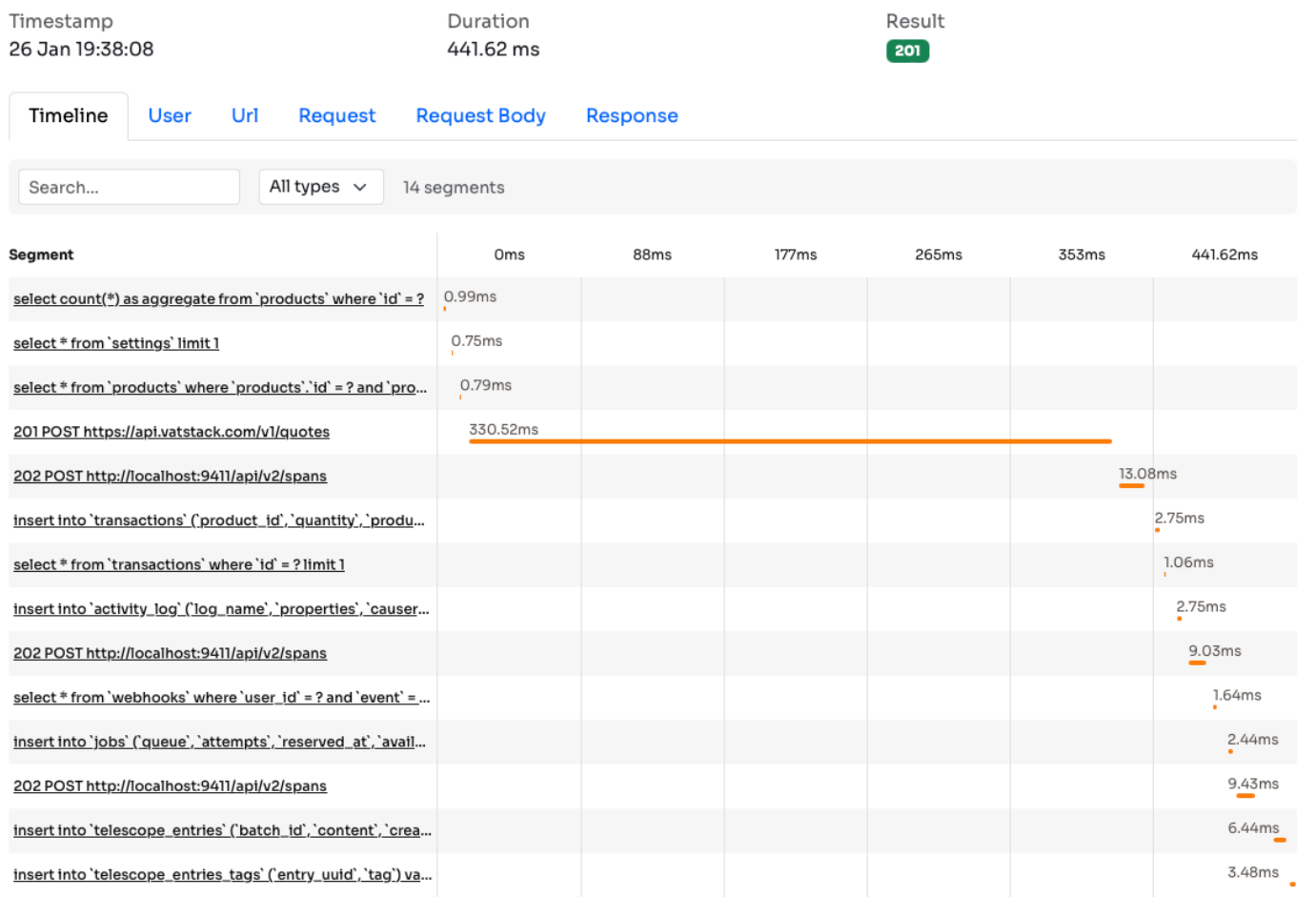
There are four spans:

- `create transaction` refers to the whole method
- `calculate vat` tracks the communication with VatStack
- `insert transaction` tracks the DB query
- Finally, `SendWebhookJob` was recorded by the package automatically. It tracks every queue job by default and puts them into the right trace. It's a great feature of the Spatie package.

Unfortunately, it's not perfect. You can see the duration is `1.307s` in the upper left corner that refers to the duration of the whole trace. But it's not true since the operation took only 399ms + 78ms for the job. Since the job is async there's a delay in dispatching it and the start of the execution by the worker process. I honestly don't know how we can overcome this problem.

In retrospect, here's the same endpoint in Inspector:

POST /api/transactions



The duration is much better and I think the timeline is also better. Of course, it's more detailed. If these small 5ms segments are annoying I have good news. You can group them using segments:

```
$vat = $vatService->calculateVat($total, $request->ip());

inspector()->addSegment(function () {
    $feeRate = $setting->fee_rate;
    $feeAmount = $total->multiply((string) $feeRate->value);
    //...
}, 'process', 'Group #1');
```

`Group #1` will be displayed in Inspector as the name of this segment. Instead of 10 small segments, you'll see only one. It's a great feature if you're in the middle of debugging and you want to see less stuff to have a better overview of your endpoint.

To sum it up:

- OpenTelemetry is a great tool for profiling your apps
- You have to add your own spans which gives you great customization
- On the other hand, you have to invest some time upfront
- It's free. However, you still need to deploy ZipKin somewhere

Compare it to Inspector:

- You just install a package
- You'll see every detail
- You can still add your own segments to customize the default behavior
- But of course, it's a paid service

XDebug + qcachegrind

The next profiling tool is the lowest level of all. It might not be the most useful but I felt I had to include at least one low-level tool.

XDebug can be used in two ways:

- Step debugging your code
- Profiling your code

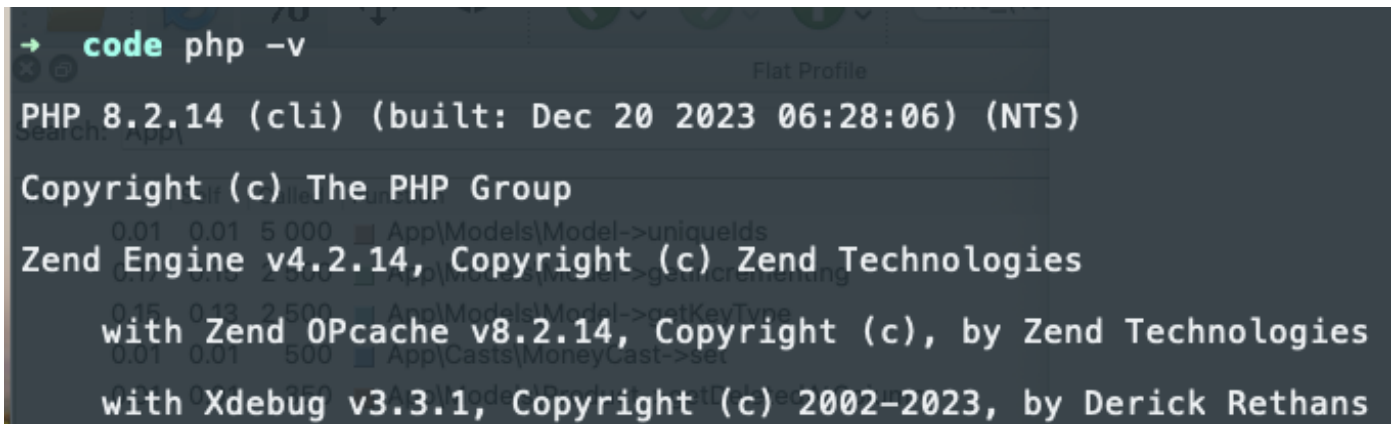
Lots of people know about the step debugging aspect of it. And it's great. I think you should set it up and use it. Here's a [Jeffrey Way video](#) that teaches you the whole process in 11 minutes and 39 seconds.

The other feature of XDebug is profiling. When you send a request to your app it can profile every method call and create a pretty big data structure out of it that can be viewed and analyzed for performance problems. The program that allows you to view these structures is called `qcachegrind` on Mac and `kcachegrind` on Linux.

You can install these two programs on Mac by running these:

```
pecl install xdebug
brew install qcachegrind
```

If you run `php -v` you should see something like this:



```
→ code php -v
PHP 8.2.14 (cli) (built: Dec 20 2023 06:28:06) (NTS)
Copyright (c) The PHP Group
Zend Engine v4.2.14, Copyright (c) Zend Technologies
with Zend OPcache v8.2.14, Copyright (c), by Zend Technologies
with Xdebug v3.3.1, Copyright (c) 2002-2023, by Derick Rethans
```

After that we need to configure XDebug in `php.ini`:

```
zend_extension=xdebug
xdebug.profiler_enabled=1
xdebug.mode=profile,debug
xdebug.profiler_output_name=cachegrind.out.%c
xdebug.start_upon_error=yes
xdebug.client_port=9003
xdebug.client_host=127.0.0.1
```

`xdebug.mode=profile` makes it listen to our requests and create a function call map.

`xdebug.profiler_output_name` is the file that it creates in the `/var/tmp` directory. `client_port` and `client_host` are only needed to step debugging.

If you're not sure where is your `php.ini` file run this:

```
php --ini
```

```
→ code php --ini
Configuration File (php.ini) Path: /opt/homebrew/etc/php/8.2
Loaded Configuration File:      /opt/homebrew/etc/php/8.2/php.ini
Scan for additional .ini files in: /opt/homebrew/etc/php/8.2/conf.d
Additional .ini files parsed:    /opt/homebrew/etc/php/8.2/conf.d/ext-opcache.ini,
/opt/homebrew/etc/php/8.2/conf.d/ext-xdebug.ini
```

As you can see, I didn't add the XDebug config in the `php.ini` file but created an `ext-xdebug.ini` file in the `conf.d` folder that is automatically loaded by PHP.

Now you need to restart your `php artisan serve`, or Docker container, or local fpm installation. If you did everything right `phpinfo()` should include the XDebug extension.

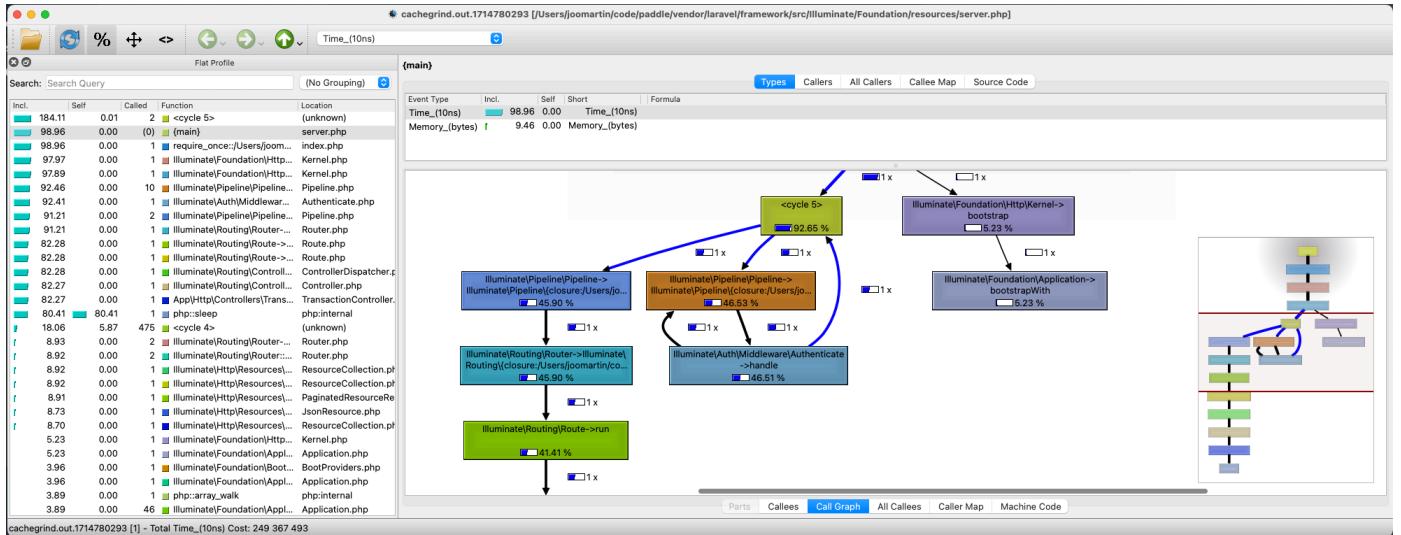
Now all you need to do is send a request to your application. After that, you should see a new file inside the `/var/tmp` directory:

```
→ tmp ls -lhta
total 41200
-rw-r--r--  1 joomartin  wheel   19M Jan 26 13:22 cachegrind.out.1714780293
drwxrwxrwt  9 root      wheel   288B Jan 26 13:22 .
drwxr-xr-x  4 _softwareupdate wheel  128B Jan 18 21:22 SoftwareUpdateCore
drwxr-xr-x  3 _accessoryupdater wheel   96B Jan 18 21:22 aud
drwxr-xr-x 34 root      wheel  1,1K Jan 18 18:29 ..
-rw-r--r--  1 _windowserver wheel   32K Jan 18 18:21 cbrgbc_1.sqlite-shm
drwxr-xr-x  3 root      wheel   96B Jan 11 04:53 com.apple.wifianalyticsd
-rw-r--r--  1 _windowserver wheel    0B Jan 11 04:53 cbrgbc_1.sqlite-wal
-rw-r--r--  1 _windowserver wheel   84K Jan 11 04:53 cbrgbc_1.sqlite
```

Let's open it:

```
qcachegrind cachegrind.out.1714780293
```

We see something like that:

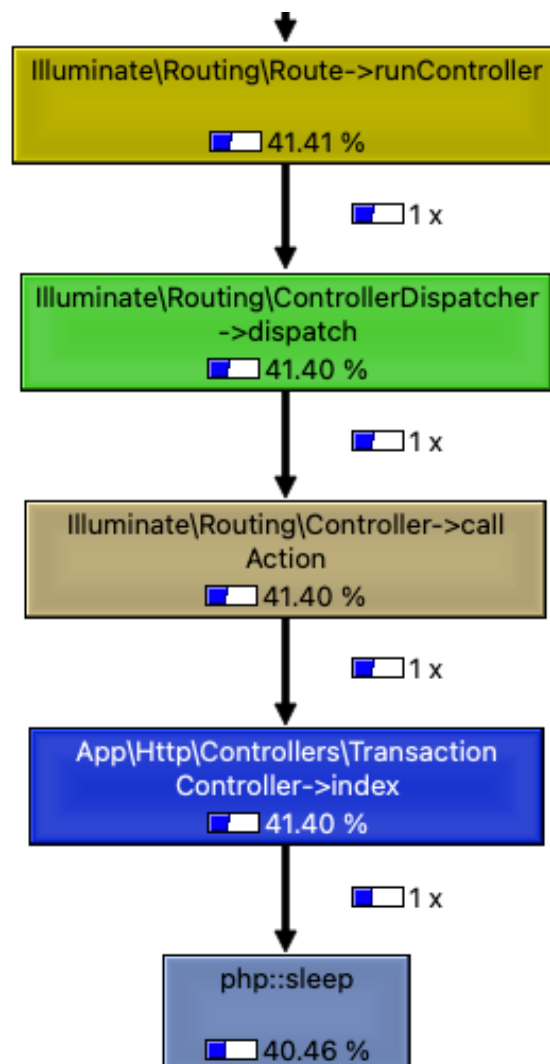


It's a little bit old school, it's a little ugly, but it's actually pretty powerful.

On the left side, you see every function that was invoked during the requests.

On the right side, you see the full call graph.

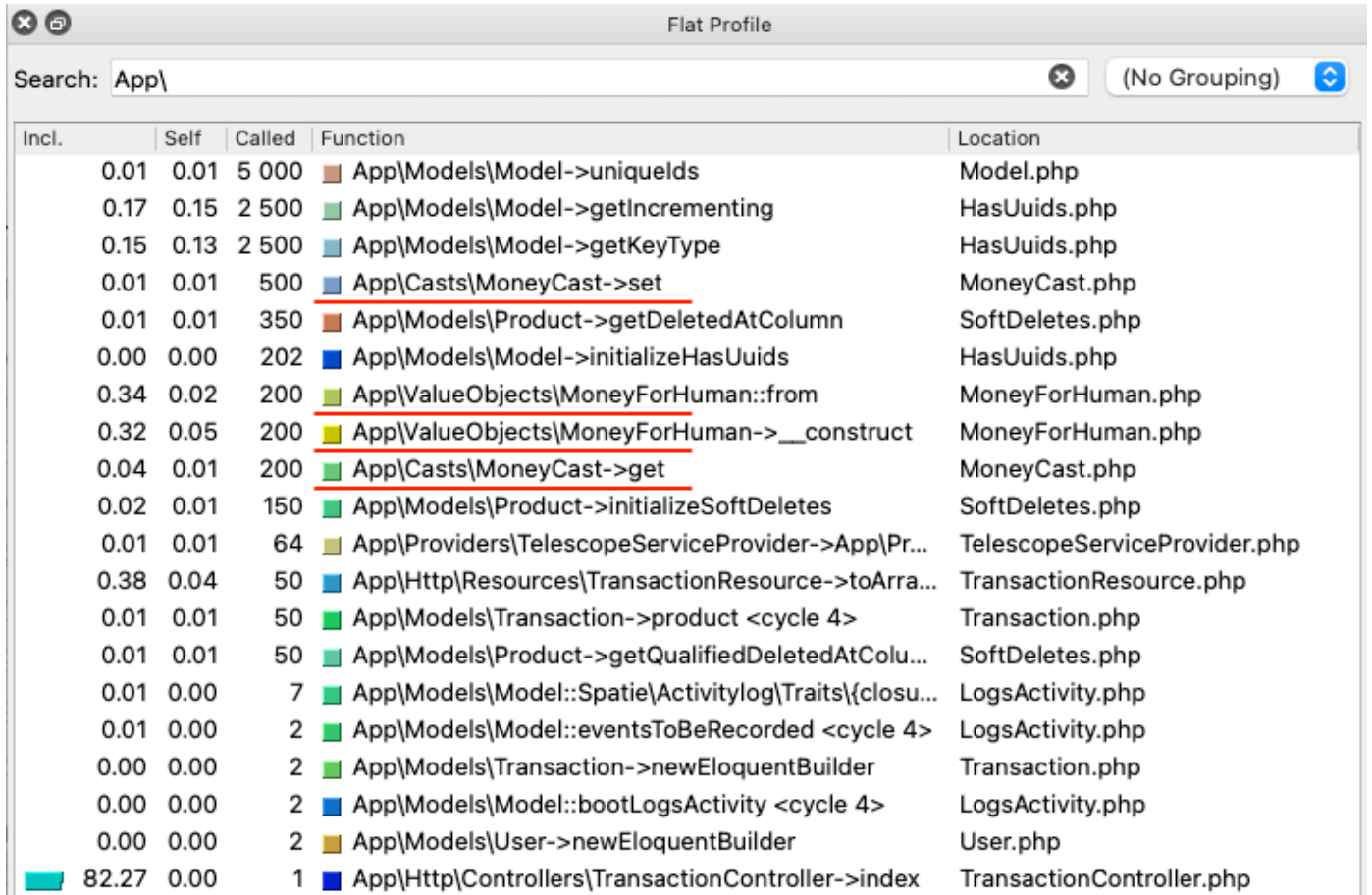
The great thing about the call graph is that it includes the time spent in the given function. Take a look at this:



This is the part where Laravel dispatches my `TransactionController` class and calls the `index` method in which I put a `sleep` function. 40% of the time was spent in the `sleep` function which is expected in this case.

The great thing about XDebug+qcachegrind is that you can really dig deep into your application's behavior. However, I think in most cases it's unnecessary. With Telescope or Inspector, you'll get a pretty great overview of your performance problems. In a standard, high-level, "business" application your problems will be most likely related to the database, and Telescope or Inspector are just better tools to profile these kinds of problems.

However, XDebug+qcachegrind can teach us a few things. For example, I never realized this:



Incl.	Self	Called	Function	Location
0.01	0.01	5 000	App\Models\Model->uniqueIds	Model.php
0.17	0.15	2 500	App\Models\Model->getIncrementing	HasUuids.php
0.15	0.13	2 500	App\Models\Model->getKeyType	HasUuids.php
0.01	0.01	500	App\Casts\MoneyCast->set	MoneyCast.php
0.01	0.01	350	App\Models\Product->getDeletedAtColumn	SoftDeletes.php
0.00	0.00	202	App\Models\Model->initializeHasUuids	HasUuids.php
0.34	0.02	200	App\ValueObjects\MoneyForHuman::from	MoneyForHuman.php
0.32	0.05	200	App\ValueObjects\MoneyForHuman->__construct	MoneyForHuman.php
0.04	0.01	200	App\Casts\MoneyCast->get	MoneyCast.php
0.02	0.01	150	App\Models\Product->initializeSoftDeletes	SoftDeletes.php
0.01	0.01	64	App\Providers\TelescopeServiceProvider->App\Pr...	TelescopeServiceProvider.php
0.38	0.04	50	App\Http\Resources\TransactionResource->toArra...	TransactionResource.php
0.01	0.01	50	App\Models\Transaction->product <cycle 4>	Transaction.php
0.01	0.01	50	App\Models\Product->getQualifiedDeletedAtColu...	SoftDeletes.php
0.01	0.00	7	App\Models\Model::Spatie\Activitylog\Traits\{closu...	LogsActivity.php
0.01	0.00	2	App\Models\Model::eventsToBeRecorded <cycle 4>	LogsActivity.php
0.00	0.00	2	App\Models\Transaction->newEloquentBuilder	Transaction.php
0.00	0.00	2	App\Models\Model::bootLogsActivity <cycle 4>	LogsActivity.php
0.00	0.00	2	App\Models\User->newEloquentBuilder	User.php
82.27	0.00	1	App\Http\Controllers\TransactionController->index	TransactionController.php

These are the functions that were executed during the requests. I highlighted four of them:

- `MoneyCast::set` was called 500 times
- `MoneyForHuman::from` was called 200 times
- `MoneyForHuman::__construct` was called 200 times
- `MoneyCast::get` was called 200 times

I give you some context. These examples come from a financial app. The request I was testing is the `GET /api/transactions`. It returns 50 transactions. A transaction record looks like this:

ID	product_id	quantity	revenue
1	1	2	1800
2	1	1	900
3	2	1	2900

It contains a sale of a product. It has other columns as well.

The `Transaction` model uses some value object casts:

```
class Transaction extends Model
{
    use HasFactory;

    protected $casts = [
        'quantity' => 'integer',
        'revenue' => MoneyCast::class,
        'fee_rate' => PercentCast::class,
        'fee_amount' => MoneyCast::class,
        'tax_rate' => PercentCast::class,
        'tax_amount' => MoneyCast::class,
        'balance_earnings' => MoneyCast::class,
        'product_data' => 'array',
    ];
}
```

`MoneyCast` is just a `Cast` that uses the `Money` value object from the [money.php](https://github.com/moneyphp/money) package:

```
class MoneyCast implements CastsAttributes
{
    public function get(Model $model, string $key, mixed $value, array $attributes): mixed
    {
        return Money::USD($value);
    }

    public function set(Model $model, string $key, mixed $value, array $attributes): mixed
    {
        return $value->getAmount();
    }
}
```

Pretty simple. The database stores scalar values and this `cast` casts them into value objects.

The `TransactionController` return with `TransactionResource` objects:

```
class TransactionResource extends JsonResource
{
    public function toArray(Request $request): array
    {

```

```

return [
    'uuid' => $this->uuid,
    'quantity' => $this->quantity,
    'revenue' => MoneyForHuman::from($this->revenue)->value,
    'fee' => MoneyForHuman::from($this->fee_amount)->value,
    'tax' => MoneyForHuman::from($this->tax_amount)->value,
    'balance_earnings' => MoneyForHuman::from($this->balance_earnings)->value,
    'customer_email' => $this->customer_email,
];
}
}

```

There are those `MoneyForHuman` calls. It's just another value object that formats `Money` objects.

The important part is that the Controller returns only 50 transactions:

```

return TransactionResource::collection(
    $request->user()->transactions()->paginate(50),
);

```

Returning only 50 transactions resulted in 1,100 calls to these objects and functions!

It's crazy. If I put something in one of these classes that takes only 50ms the whole request will take an extra **5,500ms** to complete. **That is an extra 55 seconds.**

Let's try it out!

These are the base results without slowing down the functions:

Document Path:	/api/transactions		± Martin Joo *
Document Length:	16101 bytes	11	public function
Concurrency Level:	1	12	{
Time taken for tests:	0.279 seconds	13	return [
Complete requests:	1	14	'uuid'
Failed requests:	0	15	'product'
Total transferred:	16473 bytes	16	'title'
HTML transferred:	16101 bytes	17	'url'
Requests per second:	3.59 [# /sec] (mean)	18	],
Time per request:	278.529 [ms] (mean)	19	'product'
Time per request:	278.529 [ms] (mean, across all concurrent requests)	20	'
Transfer rate:	57.76 [Kbytes/sec] received	21	]

I sent only one request and it took 278ms to complete. Of course, it will vary but it's good enough.

And now I put 3 `usleep(55000);` calls in the code:

```

class MoneyForHuman
{

```

```

public function __construct(private readonly Money $money)
{
    usleep(55000);
    // ...
}
}

class MoneyCast implements CastsAttributes
{
    public function get(): mixed
    {
        usleep(55000);
        // ...
    }

    public function set(): mixed
    {
        usleep(55000);
        // ...
    }
}

```

At the first try, `ab` timeout was exceeded which is 30 seconds by default:

```

This is ApacheBench, Version 2.3 <$Revision: 1903618 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 127.0.0.1 (be patient)...apr_pollset_poll: The timeout specified has expired (70007)

```

Let's increase it:

```

ab -n 1 -s 300 -H "Authorization: Bearer 5|JkQOThREkfVgcviCdfEEAU74WRyGHolZuKujG4fA"
http://127.0.0.1:8000/api/transactions

```

And the results are:

Document Path:	/api/transactions
Document Length:	16101 bytes
Concurrency Level:	1
Time taken for tests:	53.479 seconds
Complete requests:	1
Failed requests:	0
Total transferred:	16473 bytes
HTML transferred:	16101 bytes
Requests per second:	0.02 [#/sec] (mean)
Time per request:	53479.007 [ms] (mean)
Time per request:	53479.007 [ms] (mean, across all concurrent requests)
Transfer rate:	0.30 [Kbytes/sec] received

The request took 53.5 seconds to complete.

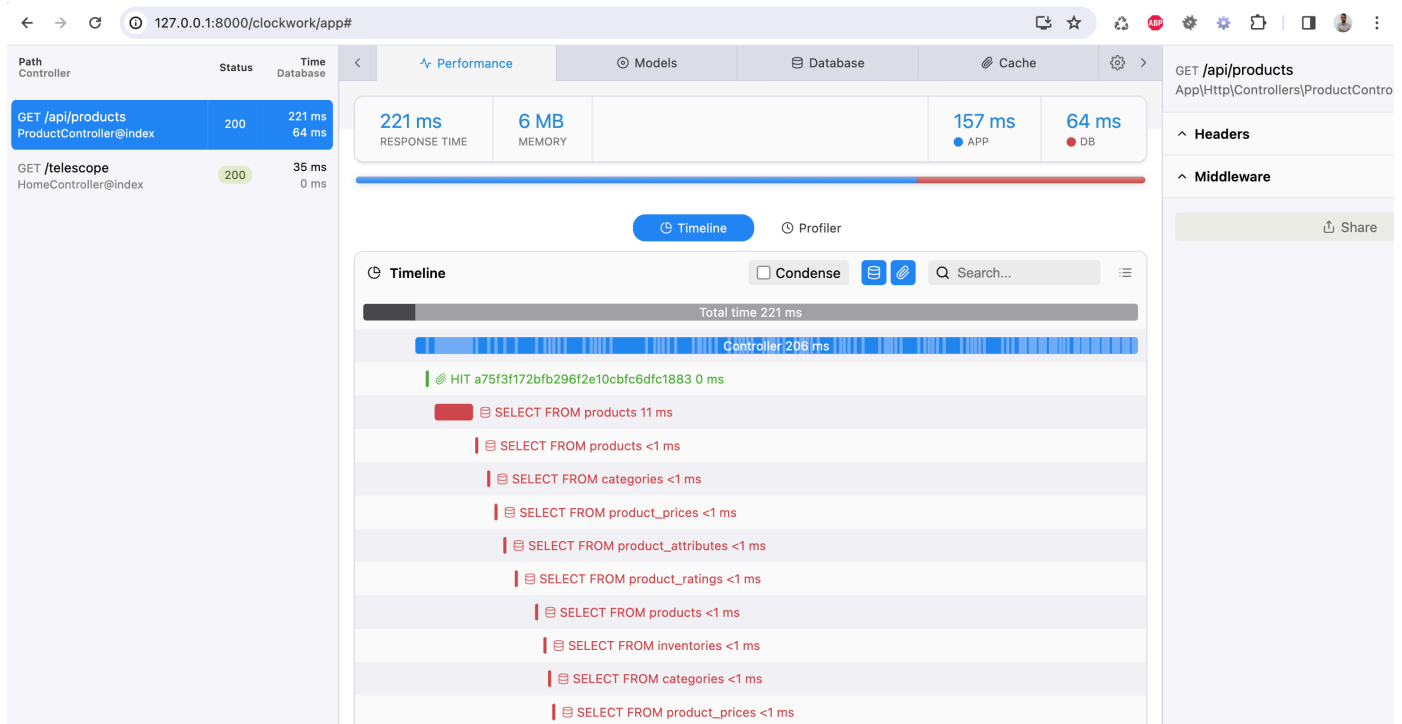
So even though XDebug+qcache grind can be a little bit too low-level for 95% of our usual performance problems as you can see they can help us to see the small details that can ruin the performance of our applications in some cases.

If you want to learn more about XDebug+qcache grind check out [this live stream](#) from the creator of XDebug.

Clockwork

There are some other useful tools to profile your applications. Clockwork and Debugbar are great examples.

Clockwork is very similar to Telescope. It's a composer package that you can install and after that, you can open `127.0.0.1:8000/clockwork/app#` and you'll get a page such as this:



It's the timeline of an API request showing all the database queries that were executed.

You can also check how many models are being retrieved to serve the request:

Performance

Models

Database

Cache

1134

RETRIEVED

Models

Search...

	Model	Retrieved	Created	Updated	Deleted
	Product	132	-	-	-
	Category	36	-	-	-
	ProductPrice	12	-	-	-
	ProductAttribute	121	-	-	-
	ProductRating	833	-	-	-

The great thing about Clockwork is that it also comes with a Chrome plugin. So you can see everything in your developer tool:

The screenshot shows a web browser at `127.0.0.1:8000/api/products` displaying a JSON response for a product. The JSON includes fields like `slug`, `title`, `description`, and `category`. Below the browser, the Laravel Clockwork performance profiler is open, showing a detailed breakdown of the request. The profiler indicates a total response time of 285 ms, with 183 ms spent in the application and 102 ms in the database. The database query `SELECT FROM products` took 10 ms and generated 99 events. The profiler also shows a `HIT` for a cache key and a `SELECT FROM product_ratings` query taking 5 ms.

```
8      "slug": "optio-quia-qui-error-praesentium-accusamus-molestias-ab",
9      "title": "suscipit sit nemo eius voluptates minima aut corrupti velit reprehenderit",
10     "description": "Minima delectus voluptatem officia earum non rerum corporis. Et tempore omnis nisi sint. Qui reprehenderit odio et
    voluptas facilis eligendi magnam minima. Fugit repellat illum ipsam aliquid. Explicabo voluptas reiciendis ut blanditiis reiciendis omnis
    doloribus. Placeat consequatur hic dolor tempora eum totam. Quis nobis recusandae veritatis labore qui.",
11     "category": {
12       "id": 30,
13       "title": "modi dolores sed",
14       "breadcrumb": [
15         "sed illo odio",
16         "nihil eum velit"
17       ]
18     }
19   }
20 }
```

Path	Controller	Status	Time	Database
GET /api/products	ProductController@index	200	291 ms	98 ms
GET /api/products	ProductController@index	200	285 ms	102 ms

Performance Summary:

- RESPONSE TIME: 285 ms
- MEMORY: 6 MB
- APP: 183 ms
- DB: 102 ms

Timeline:

- Total time 285 ms
- Controller 259 ms
- HIT a75f3f172bfb296f2e10cbfc6dfc1883 0 ms
- SELECT FROM products 10 ms (99 events)
- SELECT FROM product_ratings 5 ms (10 events)

Right Panel:

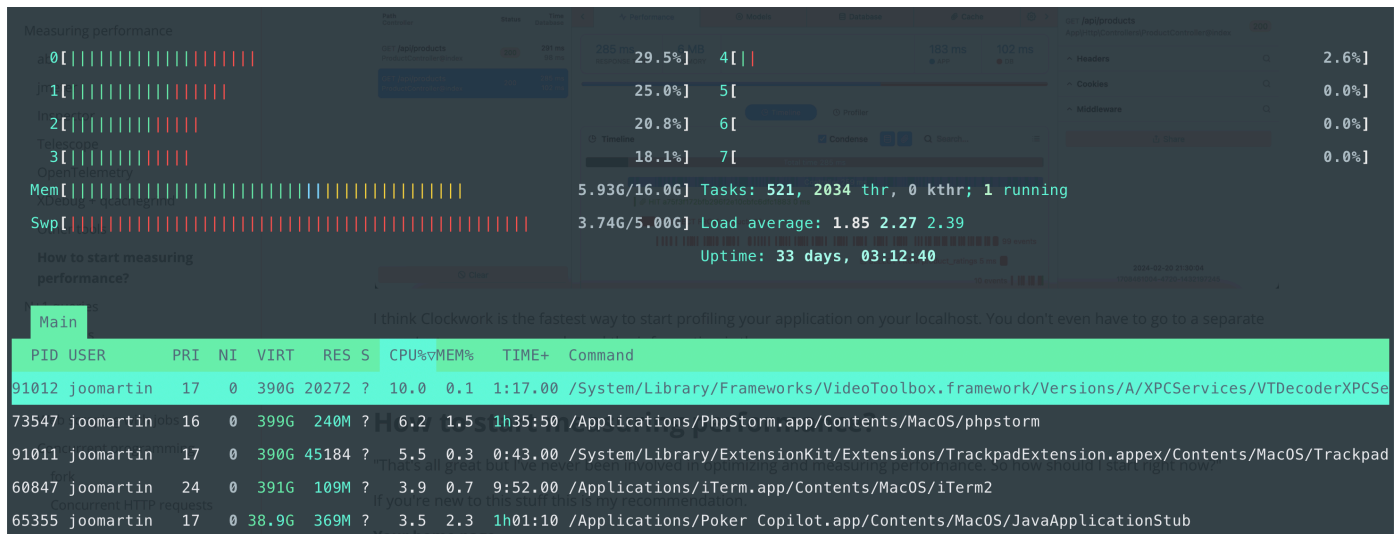
- GET /api/products (200)
- App\Http\Controllers\ProductController@index
- Headers
- Cookies
- Middleware

2024-02-20 21:30:04
1708461004-4720-1432197245

I think Clockwork is the fastest way to start profiling your application on your localhost. You don't even have to go to a separate page. Just open your console and the information is there.

htop

The last tool I'd like to talk about is `htop`. It's a simple but very powerful command line tool that I'll use in the rest of this book. It looks like this:



You can check the utilization of your CPU and memory. It's a very important tool for debugging performance issues in real-time. By real-time I mean two things:

- When shit happens and there is some serious performance issue in your production environment you can check out `htop` and see what's happening real-time.
- When you're developing a feature on your local machine you can always check `htop` to get an idea about the CPU load. Of course, it's highly different from your prod servers but it can be a good indicator.

Other than the visual representation of the load of the cores we can also see the load average numbers. They are 1.85, 2.27, and 2.39 in my case. These numbers represent the overall load of your CPU. The three numbers mean:

- 1.85 (the first one) is the load average of the last 1 minute
- 2.27 (the second one) is the load average of the last 5 minutes
- 2.39 (the last one) is the load average of the the last 15 minutes

So we have an overview of the last 15 minutes.

What does a number such as 1.85 actually mean? It means that the overall CPU utilization was around 23% on my machine for the last 1-minute period. Straightforward, right?

If you have only 1 CPU core a load average of 1 means your core is working 100%. It is fully utilized. If your load average is 2 then your CPU is doing twice as much work as it can handle.

But if you have 2 CPU cores a load average of 1 means your cores are working at 50%. In this case, a load average of 2 means 100% utilization.

So the general rule is that if the load average is higher then the number of cores your server is overloaded.

Back to my example. I have 8 cores so a load average of 8 would be 100% utilization. My load average is 1.85 on the image so it means $1.85/8$ or about 23% CPU load.

How to start measuring performance?

"That's all great but I've never been involved in optimizing and measuring performance. So how should I start right now?"

If you're new to this stuff this is my recommendation.

Your home page

In a typical business application where users must log in probably one of the most important pages is the dashboard, the home page that presents right after they log in. If it's a publicly available webpage than it is the landing page. If you don't know where/how to start this is the perfect place.

- Determine how many users you want to/have to serve. Let's say it's 1,000
- Open up ab or jmeter and send 1,000 requests to your home page.
 - Play with the ramp-up times
 - Play with the concurrency level
 - Come up with a reasonable goal. For example, "I want to be able to serve 100 concurrent users with a maximum load time of 1.5 seconds" (these numbers are completely random, please don't take them seriously)
- Now open up Inspector or Telescope and identify what takes a long time
- Pick up the low-hanging fruit and solve it
- Continue reading the book :)

Your most valuable feature

I know, I know... Every feature of your app is "the most important at the moment" according to the product team. However, we all know we can identify a handful of features that are the most critical in the application no matter what. Try to identify them and measure them the same way as your home page. However, in this case, your target numbers can be lower because it's usually rare that 72% of your users use the same feature at the same time. It's always true to the home page but usually it's not the case with other features. Unless, of course, your feature has some seasonality such as booking.com or it follows some deadlines such as accounting softwares. In this case, you know that on X day of every month 90% of users will use that one feature.

Your heaviest background job

We tend to forget to optimize background jobs because they run in the background and they do not overly affect the overall user experience. However, they still use our servers. They consume CPU and RAM. They cost us money.

Just as with features, try to identify your most important/critical jobs and analyze them with Inspector and/or Telescope the same way as if they were web requests. Try to reduce the number of queries, the overall memory consumption, and the execution time with techniques discussed in the book.

When you set target numbers (such as serving 100 concurrent users, loading the page within 500ms, etc) it's important to use the hardware as your production. Usually, the staging environment is a good starting point.

When you debug a specific feature or a job you can use your local environment as well. Of course, execution times will be different compared to production, but you can think in percentages. For example, "this job took 10s to finish but now it only takes 9s. I gained 10%." The number of queries, and the overall memory consumption is still be similar to production.

Database indexing

My goal in this chapter is to give you the last indexing tutorial you'll ever need. Please do not skip the following pages.

This is one of the most important topics to understand, in my opinion. No matter what kind of application you're working there's a good chance it has a database. So it's really important to understand what happens under the hood and how indexes actually work. Because of that, this chapter starts with a little bit of theory.

In order to understand indexes, first we need to understand at least 6 data structures:

- Arrays
- Linked lists
- Binary trees
- Binary search trees
- B-Trees
- B+ Trees

Arrays

They are one of the oldest data structures. We all use arrays on a daily basis so I won't go into details, but here are some of their properties from a performance point of view:

Operation	Time complexity	Space complexity
Accessing a random element	$O(1)$	$O(1)$
Searching an element	$O(N)$	$O(1)$
Inserting at the beginning	$O(N)$	$O(N)$
Inserting at the end	$O(1)$ or $O(N)$ if the array is full	$O(1)$
Inserting at the middle	$O(N)$	$O(N)$

An array is a fixed-sized contiguous data structure. The array itself is a pointer to a memory address and each subsequent element has a memory of $x + (\text{sizeof}(t) * i)$ where

- x is the first memory address where the array points at
- $\text{sizeof}(t)$ is the size of the data type. For example, an `int` takes up 8 bytes
- i is the index of the current element

This is what an array looks like:

	21	12	38	2
index	0	1	2	3
address	100	108	116	124

The subsequent memory address has an interesting implication: your computer has to shift the elements when inserting or deleting an item. This is why mutating an array in most cases is an $O(N)$ operation.

Since it's a linear data structure with subsequent elements and memory addresses searching an element is also an $O(N)$ operation. You need to loop through all the elements until you find what you need. Of course, you can use binary search if the array is sorted. Binary search is an $O(\log N)$ operation and quicksort is an $O(N * \log N)$ one. The problem is that you need to sort the array every single time you want to find an element. Or you need to keep it sorted all the time which makes inserts and deletes even worse.

What arrays are really good at is accessing random elements. It's an $O(1)$ operation since all PHP needs to do is calculating the memory address based on the index.

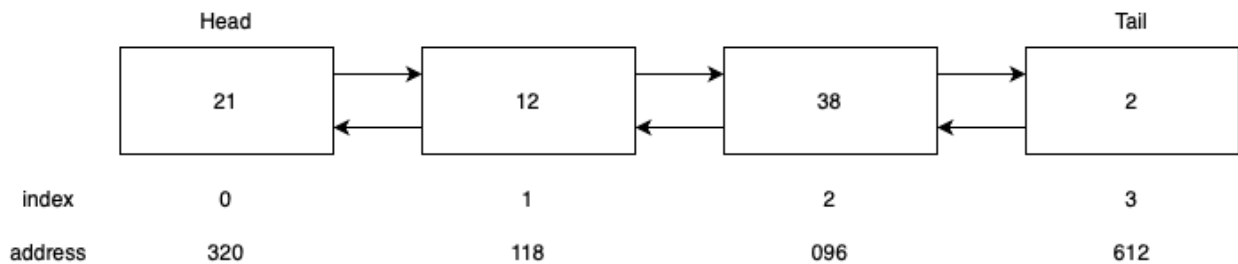
The main takeaway is that **searching and mutating is slow**.

Linked list

Since arrays have such a bad performance when it comes to inserting and deleting elements engineers came up with a linked list to solve these problems.

A linked list is a logical collection of random elements in memory. They are connected only via pointers. Each item has a pointer to the next one. There's another variation called doubly linked list where each element has two pointers: one for the previous and one for the next item.

This is what it looks like:



Memory addresses are not subsequent. This has some interesting implications:

Operation	Time complexity	Space complexity
Accessing a random element	$O(N)$	$O(1)$
Searching an element	$O(N)$	$O(1)$
Inserting at the beginning	$O(1)$	$O(1)$
Inserting at the end	$O(1)$	$O(1)$
Inserting at the middle	$O(N)$	$O(1)$

Since a linked list is not a coherent structure in memory, inserts always have a better performance compared to an array. PHP doesn't need to shift elements. It only needs to update pointers in nodes.

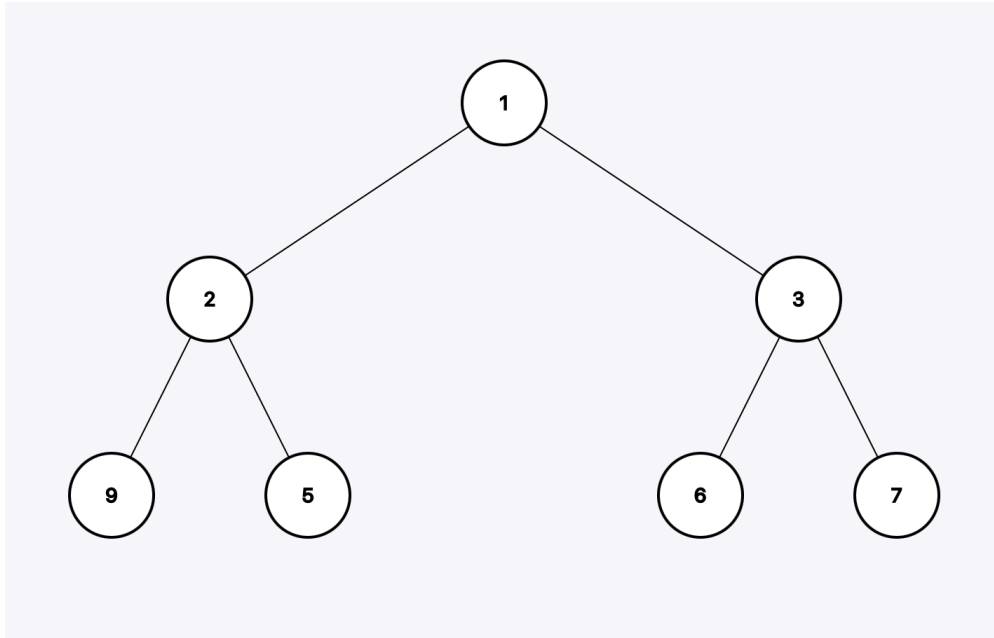
A linked list is an excellent choice when you need to insert and delete elements frequently. In most cases, it takes considerably less time and memory.

However, **searching is as slow** as it was with arrays. It's still $O(n)$.

Binary tree

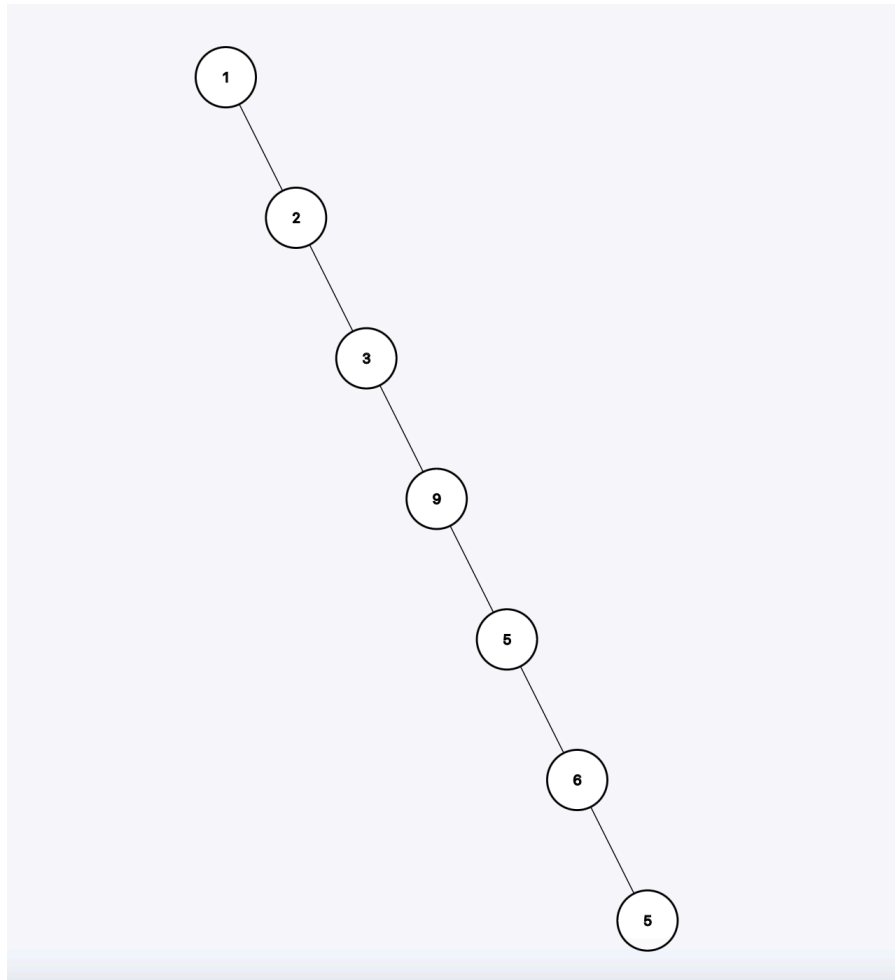
The term binary tree can be misleading since it has lots of special versions. However, a simple binary means a tree where every node has two or fewer children.

For example, this is a binary tree:



The only important property of this tree is that each node has two or fewer children.

This is also a binary tree with the same values:



Now, let's think about how much it takes to traverse a binary tree. For example, in the first tree, how many steps does it take to traverse from the root node to one of the leaf nodes (9, 5, 6, 5)? It takes three steps. If I want to go to the left-most node (9) it'd look like this (we're already at the root node):

- 2
- 9

Now let's do the same with the second tree. How many steps does it take to go to the leaf node (to 43, starting from the root)? 6 steps.

Both trees have 7 nodes. Using the first one takes only 2 steps to traverse to one of the leaf nodes but using the second one takes 6 steps. So the number of steps is not a function of the number of nodes but the height of the tree which is 2 in the first one and 6 in the second one. We don't count the root node.

Both of these trees have a name. The first one is a complete tree meaning **every** node has **exactly** two children. The second one is a degenerative tree meaning each parent has only one child. These are the two ends of the same spectrum. The first one is perfect and the other one is useless.

In a binary tree, density is the key. The goal is to represent the maximum number of nodes in the smallest depth binary tree possible.

The minimum height of a binary tree is **$\log n$** which is shown in the first picture. It has 7 elements and the height is 3.

The maximum height possible is **$n-1$** which is shown in the second picture. 7 elements with a height of 6.

From these observations we can conclude that traversing a binary tree is an $O(\log h)$ operation where h is the height of the tree.

	# of elements	Height	Time complexity
Complete tree	7	3	$O(\log n)$
Degenerate tree	7	6	$O(n-1)$

To put it in context, if you have a tree with 100,000,000 elements and your CPU can run 100,000,000 operations per seconds:

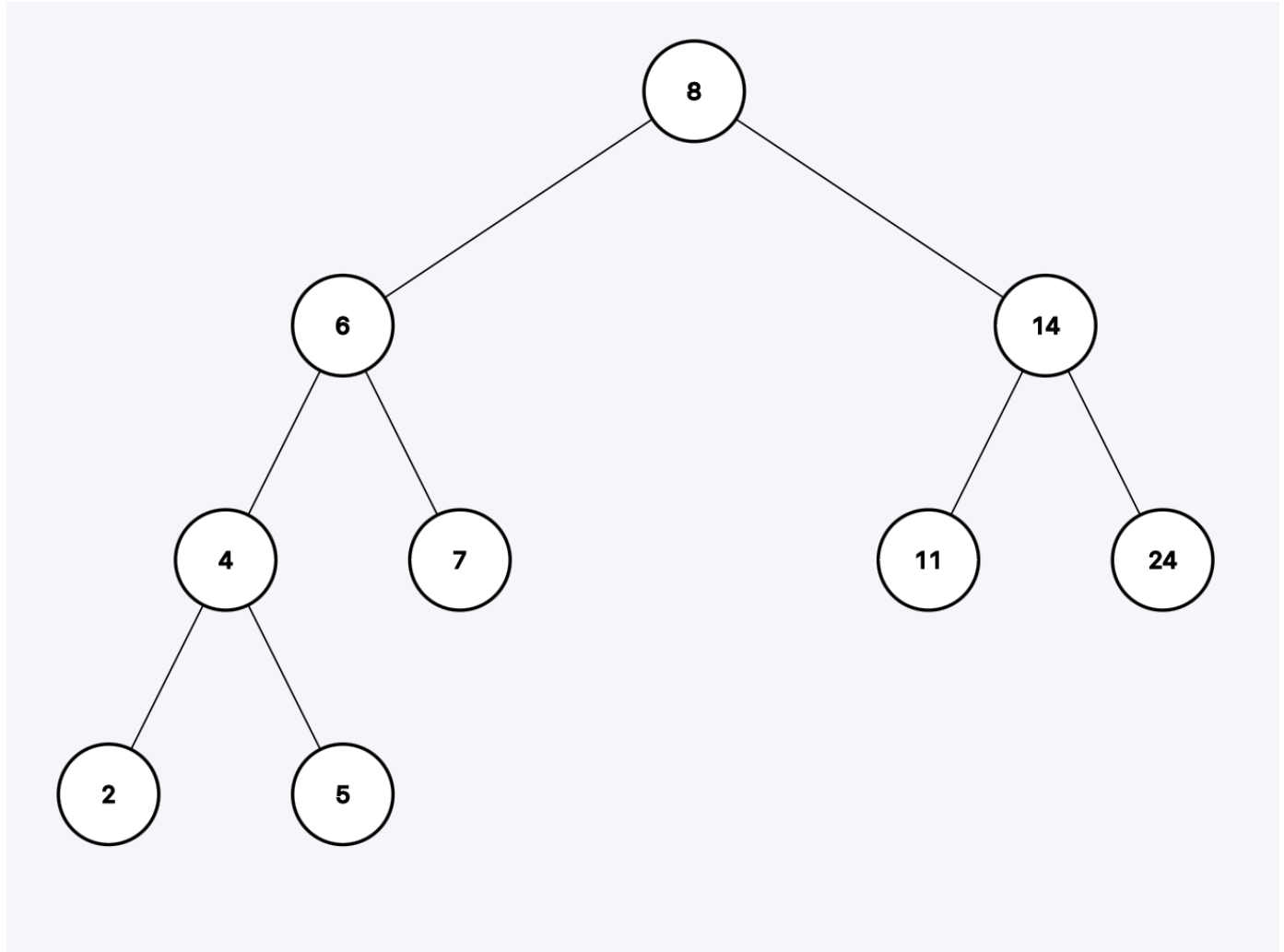
	# of iterations	Time to complete
$O(\log n)$	26	0,00000026 seconds
$O(n - 1)$	99,999,999	0,99 seconds

There's a 3,846,153 time difference between the two so engineers came up with the the following conclusion: if a tree is structured well it can traverse it in $O(\log n)$ time which is far better than arrays or linked lists.

Binary search tree (BST)

So binary trees **can** have pretty great time complexity when it comes to traversing their nodes. Can we use them to efficiently search elements in $O(\log n)$ time?

Enter the binary search tree:

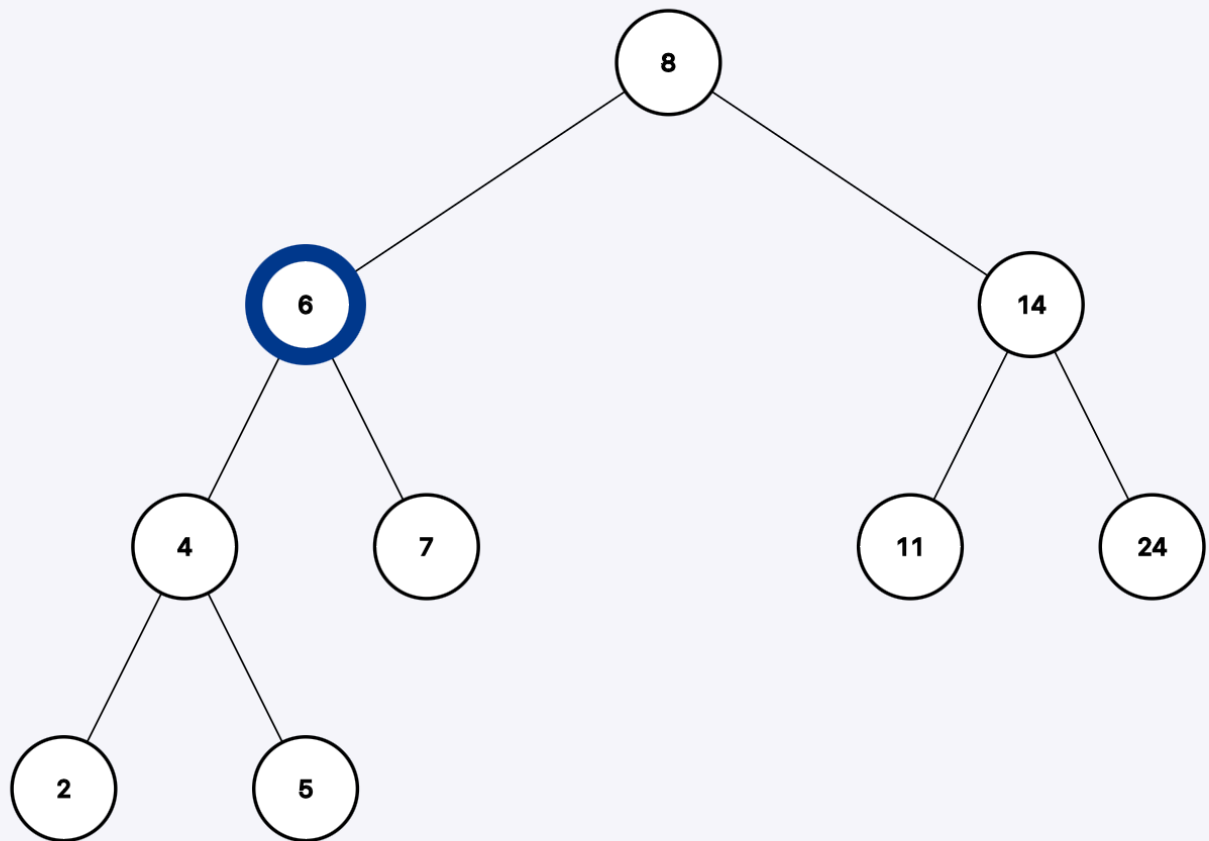


It has three important properties:

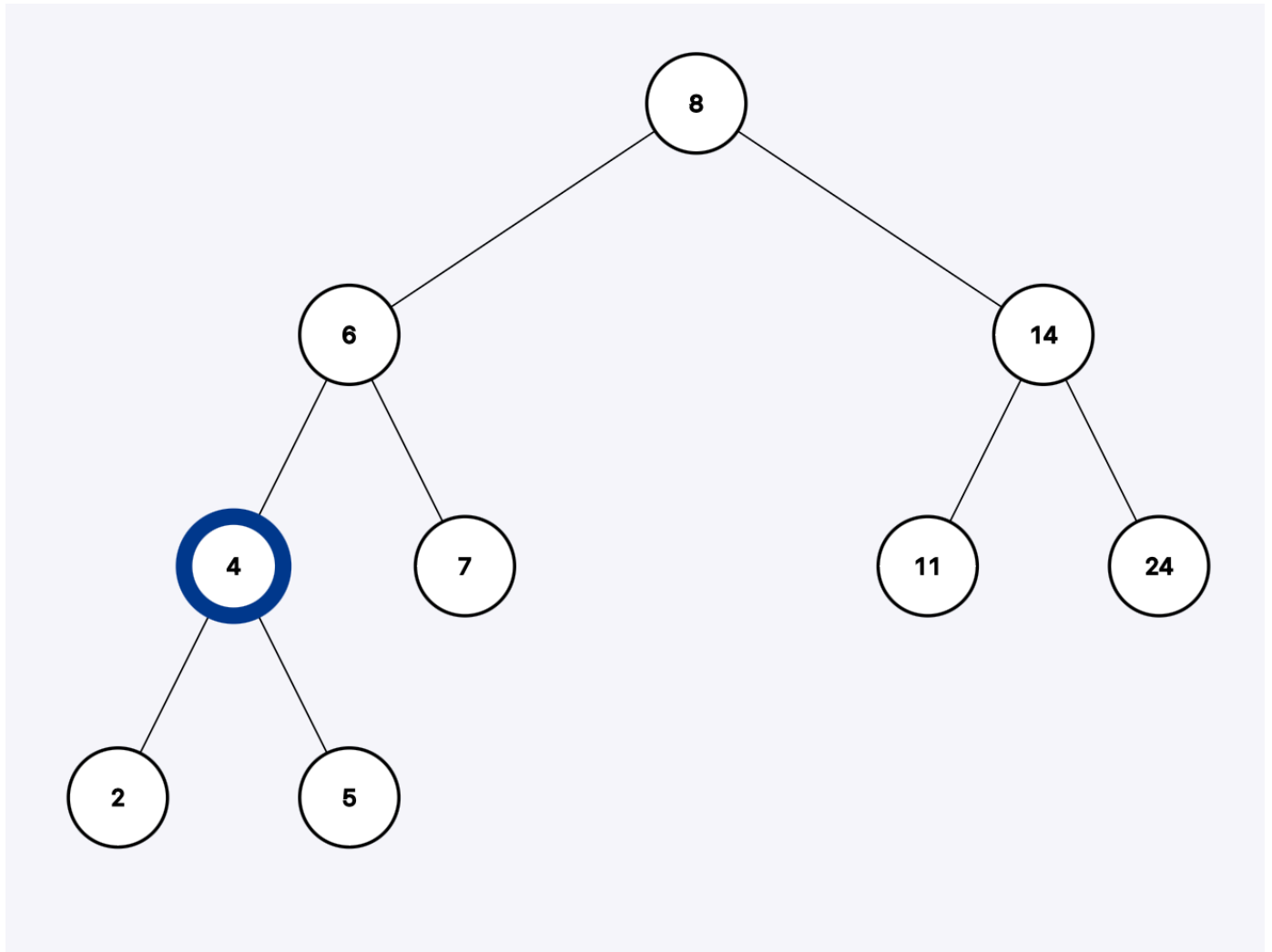
- Each node has two or fewer children
- Each node has a left child that is less than or equal to itself
- Each node has a right child that is greater than itself

The fact that the tree is ordered makes it pretty easy to search elements, for example, this is how we can find 5.

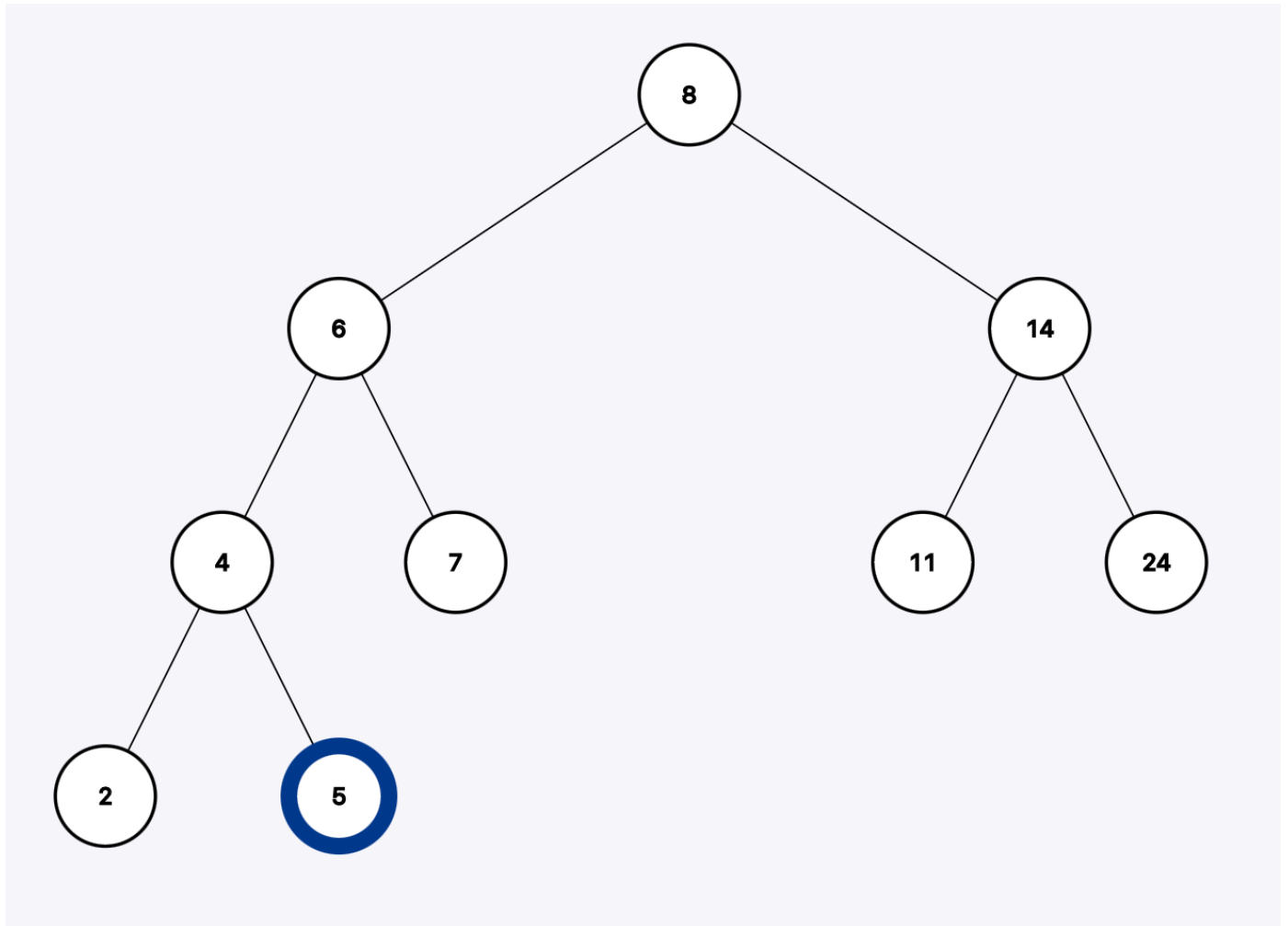
Eight is the starting point. Is it greater than 5? Yes, so we need to continue in the left subtree.



Is 6 greater than 5? Yes, so let's go to the left subtree.



Is 4 greater than 5? Nope. Each node has a right child that is greater than itself. So we go right.

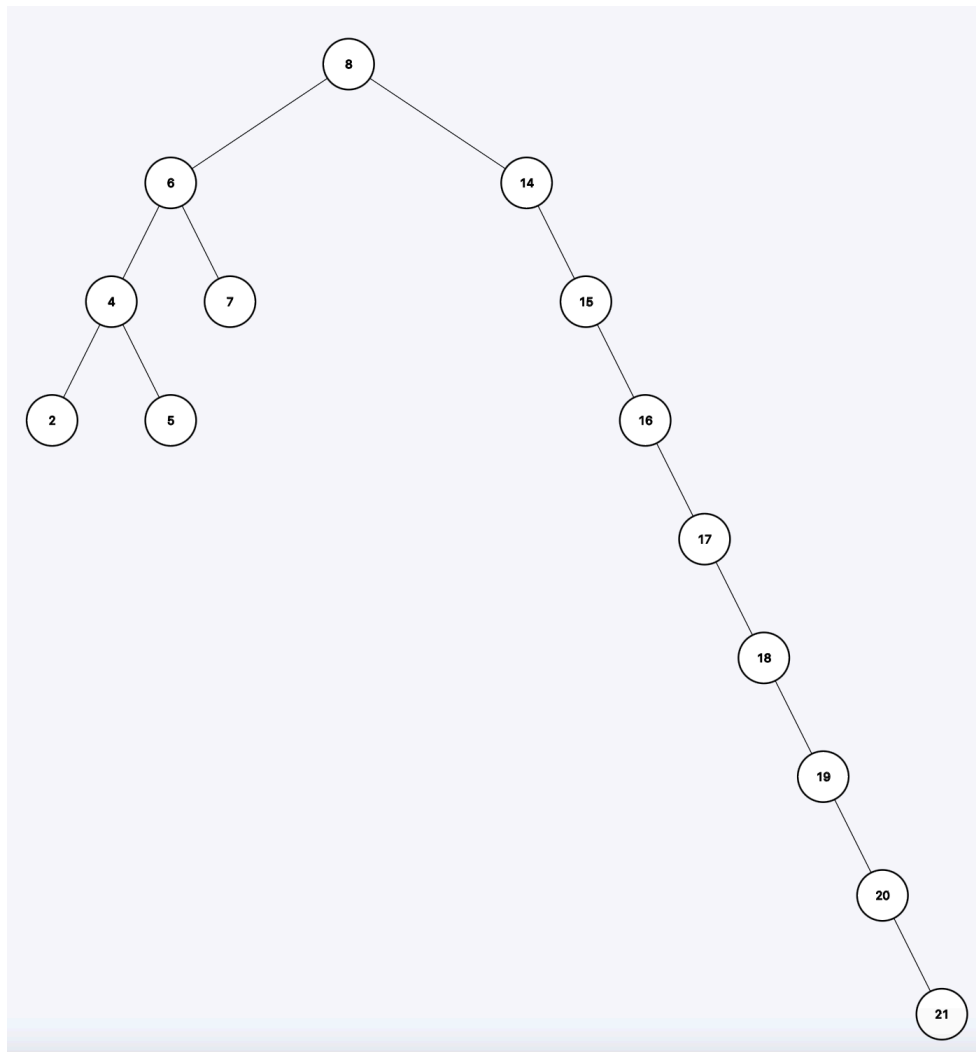


Is 5 equal to 5? Yes.

We found a leaf node in just 3 steps. The height of the tree is 3, and the total number of elements is 9. This is the same thing we discussed earlier. The cost of the search is $(O \log N)$.

So if we take a usual binary tree and add two constraints to it so it is ordered at any time we have $(O \log N)$ search.

Unfortunately, the constraints of a BST don't tell anything about balance. So this is also a perfectly fine BST:



Each node has two or fewer children. The left child is always less than or equal to the parent. The right child is always greater than the parent. But the right side of the tree is very unbalanced. If you want to find the number 21 (the bottom node in the right subtree) it becomes an $O(N)$ operation.

Binary search trees were invented in 1960, 35 years before MySQL.

Indexing in the early days

Databases and storage systems were emerging in the 1960's. The main problem was the same as today: accessing data was slow because of I/O operations.

Let's say we have a really simple `users` table with 4 columns:

ID	name	date_of_birth	job_title
1	John Doe	2005-05-15	Senior HTML programmer
2	Jane Doe	1983-08-09	CSS programmer
3	Joe Doe	1988-12-23	SQL programmer
4	James Hetfield	1969-08-03	plumber

For simplicity, let's assume a row takes 128 bytes to store on the disk. When you read something from the disk the smallest unit possible is 1 block. You cannot just randomly read 1 bit of information. The OS will return the whole block. For this example, we assume a block is 512 bytes. So we can fit 4 records ($4 * 128B$) into one block (512B). If we have 100 records we need 25 blocks.

Size of a record	128B
Size of a block	512B
# of records in one block	4
# of records overall	100
# of block needed to store the table	25

If you run the following query against this table (assuming no index, no PK):

```
select *  
from users  
where id = 50
```

Something like this happens:

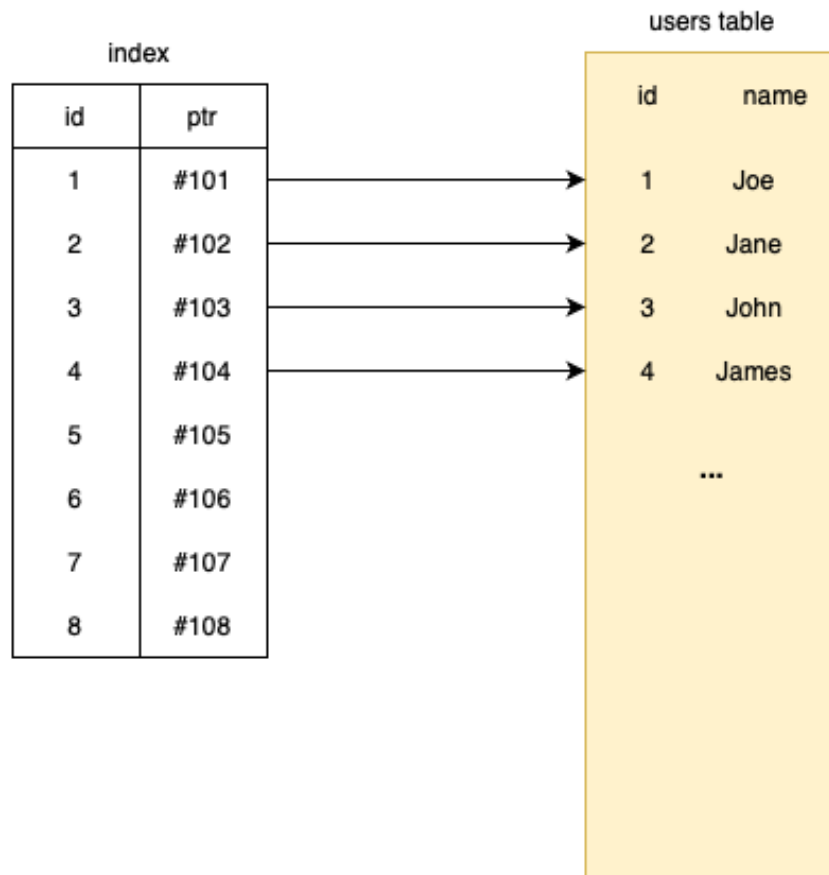
- The database will loop through the table
- It reads the first block from disk that contains row #1 - row #4
- It doesn't contain user #50 so it continues

In the worst-case scenario, it executes **25 I/O operations** scanning the table block-by-block. This is called a **full table scan**. It's slow. So engineers invented indexing.

Single-level indexing

As you can see, the problem was the size and the number of I/O operations. Can we reduce it by introducing some kind of index? Some kind of secondary table that is smaller and helps reduce I/O operations? Yes, we can.

Here's a simple index table:



The index table stores every record that can be found in `users`. They both have 100 rows. The main benefit is that the index is small. It only holds an ID that is equivalent to the ID in the `users` table and a pointer. This pointer points to the row on the disk. It's some kind of internal value with a block address or something like that. How big is this index table?

Let's assume that both the ID and ptr columns take up 8 bytes of space. So a record's size in the index table is 16 bytes.

Size of a record	16B
Size of a block	512B
# of records in one block	32
# of records overall	100
# of blocks needed to store the index table	4

Only 4 blocks are needed to store the entire index on disk. To store the entire table the number of blocks is 25. It's a 6x difference.

Now what happens when we run?

```
select *  
from users  
where id = 50
```

- The database reads the index from the disk block-by-block
- It means **4 I/O operations** in the worst-case scenario
- When it finds #50 in the index it queries the table based on the pointer which is another I/O

In the worst-case scenario, it executes **5 I/O** operations. Without the index table, it was **25**. It's a 5x performance improvement. Just by introducing a "secondary table."

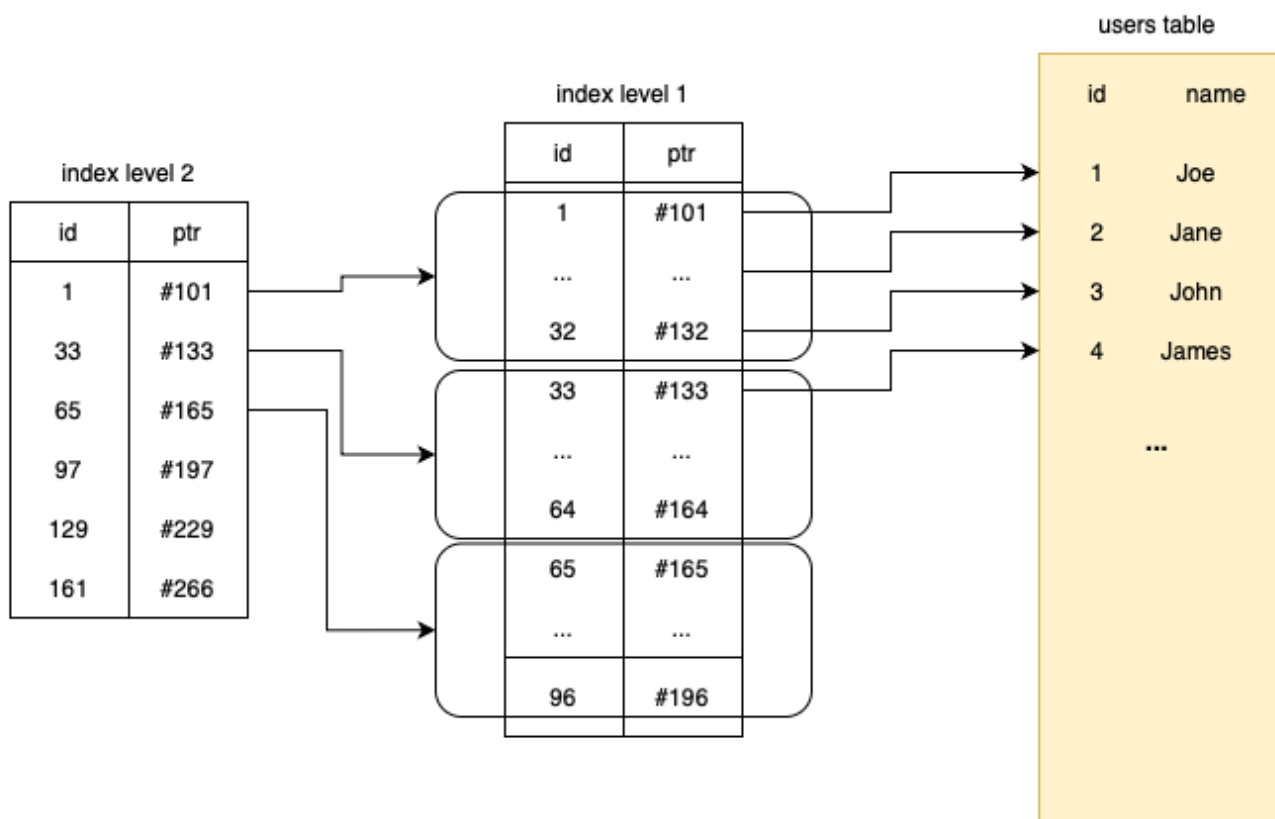
Multi-level indexing

An index table made things much better, however, the main issue remained the same: size and I/O operations. Now, imagine that the original `users` table contains 1,000 records instead of 100. This is what the I/O numbers would look like:

	# of blocks to store the data	# of I/O to query data
Database table with 1,000 users	250	250
Index table with 1,000 users	40	41

Everything is 10x larger, of course. So engineers tried to divide the problem even more by chunking the size into smaller pieces and they invented multi-level indexes. Now we said that you can store 32 entries from the index table in a single block. What if we can have a new index where every entry points to an entire block in the index table?

Well, this is a multi-level index:



Each entry in the second level index points to a range of records in the first level index:

- Row #1 in L2 points to row #1 - row #32 in L1
- Row #2 points to row #33 - row #64
- etc

Each row in L2 points to a chunk of 32 rows in L1 because that's how many records can fit into one block of disk.

If the L1 index can be stored using 40 blocks (as discussed earlier), then L2 can be stored using $40/32$ blocks. It's because in L2 every record points to a chunk of 32 records in L1. So L1 is 32x bigger than L2. 1,000 rows in L1 is 32 rows in L2.

The space requirement for L2 is $40/32$ or **2 blocks**.

	# of blocks to store the data
Database table with 1,000 users	250
L1 index with 1,000 users	40
L2 index with 1,000 users	2

What happens when we run:

```
select *
from users
where id = 50
```

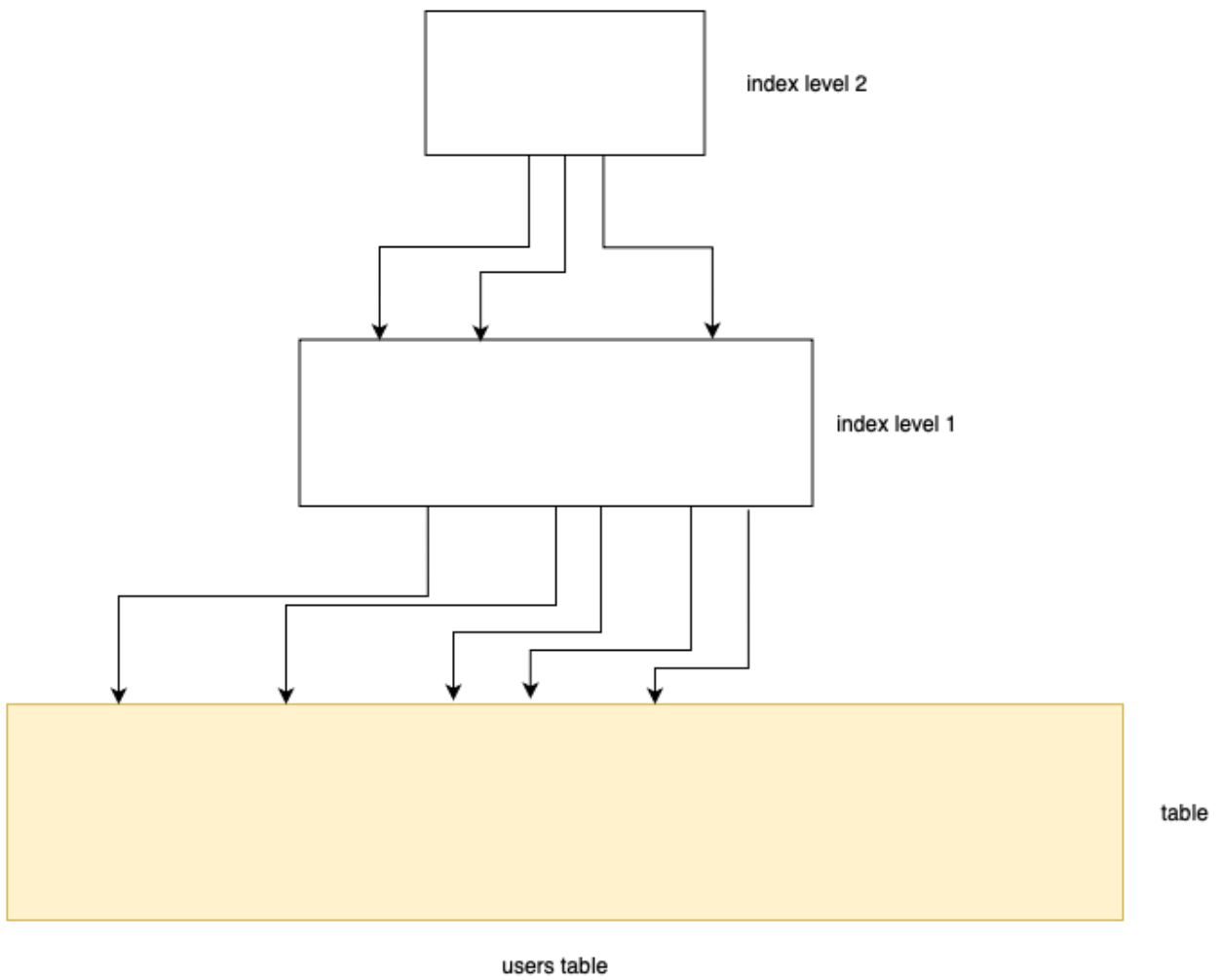
- The database reads L2 block-by-block
- In the worst-case scenario, it read **2 blocks** from the disk
- It finds the one that contains user #50
- It reads **1 block** from L1 that contains user #50
- It reads **1 block** from the table that contains user #50

Now we can find a specific row by just reading **4 blocks** from the disk.

	# of blocks to store the data	# of I/O to query data
Database table with 1,000 users	250	250
L1 index with 1,000 users	40	41
L2 index with 1,000 users	2	4

They were able to achieve a 62x performance improvement by introducing another layer.

Now let's do something crazy. Rotate the image by 90 degrees:



It's a tree! IT'S A TREE!!

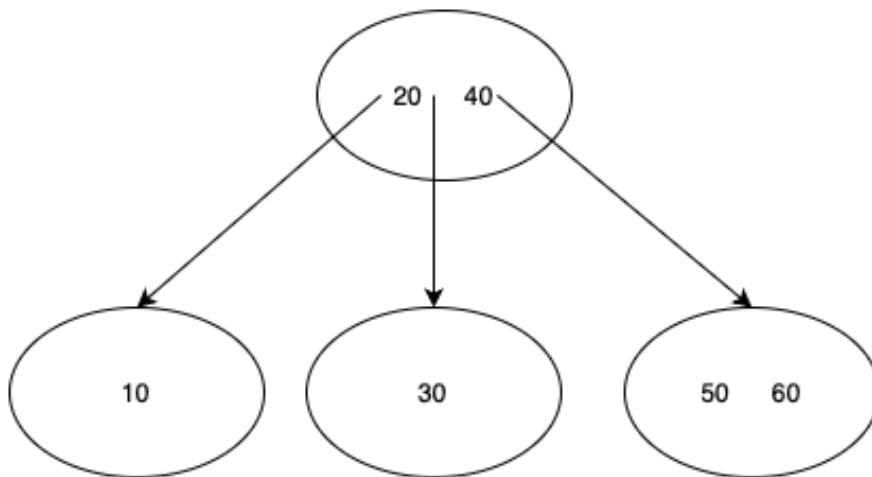
B-Tree

In 1970, two gentlemen at Boeing invented B-Trees which was a game-changer in databases. This is the era when Unix timestamps looked like this: 1 If you wanted to query the first quarter's sales, you would write this: `between 0 and 7775999`. Black Sabbath released Paranoid. Good times.

What does the **B** stand for? They didn't specify it, but often people call them "balanced" trees.

A B-Tree is a specialized version of an M-way tree. "What's an M-way tree?" Glad you asked!

This is a 3-way tree:

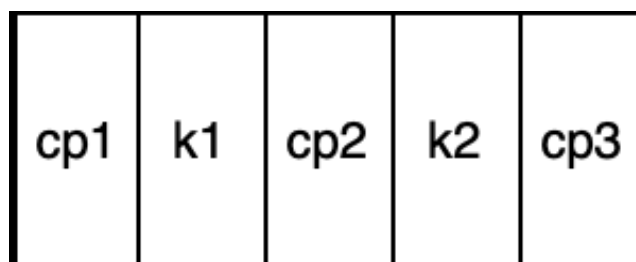


It's a bit different than a binary tree:

- Each node holds more than one value. To be precise a node can have $m-1$ values (or keys).
- Each node can have up to m children.
- The keys in each node are in ascending order.
- The keys in children nodes are also ordered compared to the parent node (such as 10 is at the left side of 20 and 30 is at the right side)

Since it's a 3-way tree a node can have a maximum of 3 children and can hold up to two values.

If we zoom in on a node it looks like this:



cp stands for child pointer and **k** stands for key.

The problem is however, there are no rules or constraints for insertion or deletion. This means you can do whatever you want, and m-way trees can become unbalanced just as we see with binary search trees. If a tree is unbalanced searching becomes $O(n)$ which is very bad for databases.

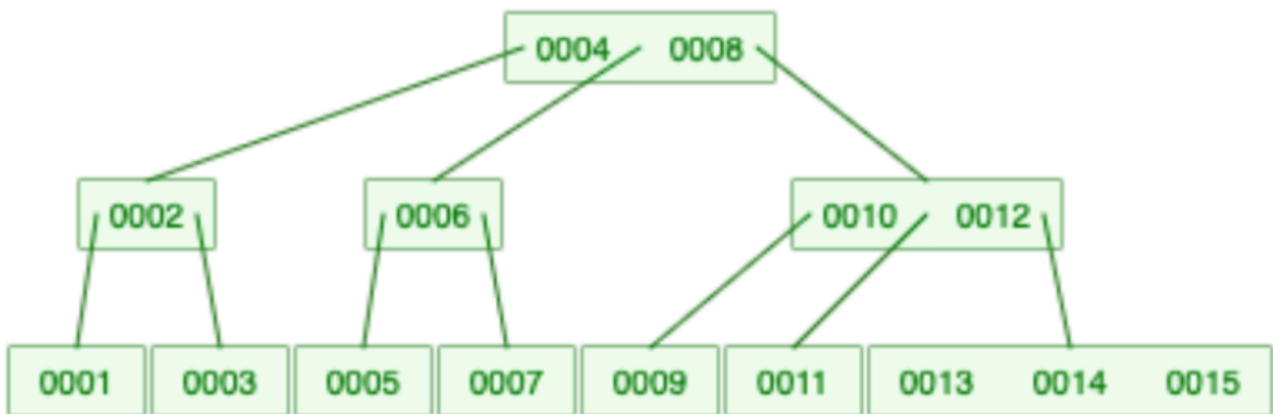
So B-Trees are an extension of m-way search trees. They define the following constraints:

- The root node has at least 2 children (or subtrees).
- Each other node needs to have at least $m/2$ children.
- All leaf nodes are at the same level

I don't know how someone can be that smart but these three simple rules make B-trees always at least half full, have few levels, and remain **perfectly balanced**.

There's a [B-Tree visualizer website](#) where you can see how insertion and deletion are handled and how the tree remains perfectly balanced at all times.

Here you can see numbers from 1 to 15 in a 4-degree B-Tree:



Of course, in the case of a database, every node has a pointer to the actual record on disk just as we discussed earlier.

The next important thing is this: MySQL **does not use a standard B-Tree**. Even though we use the word `BTREE` when creating an index it's actually a B+ Tree. It is stated in the [documentation](#):

The use of the term B-tree is intended as a reference to the general class of index design. B-tree structures used by MySQL storage engines may be regarded as variants due to sophistications not present in a classic B-tree design. - [MySQL Docs](#)

It is also said by Jeremy Cole multiple times:

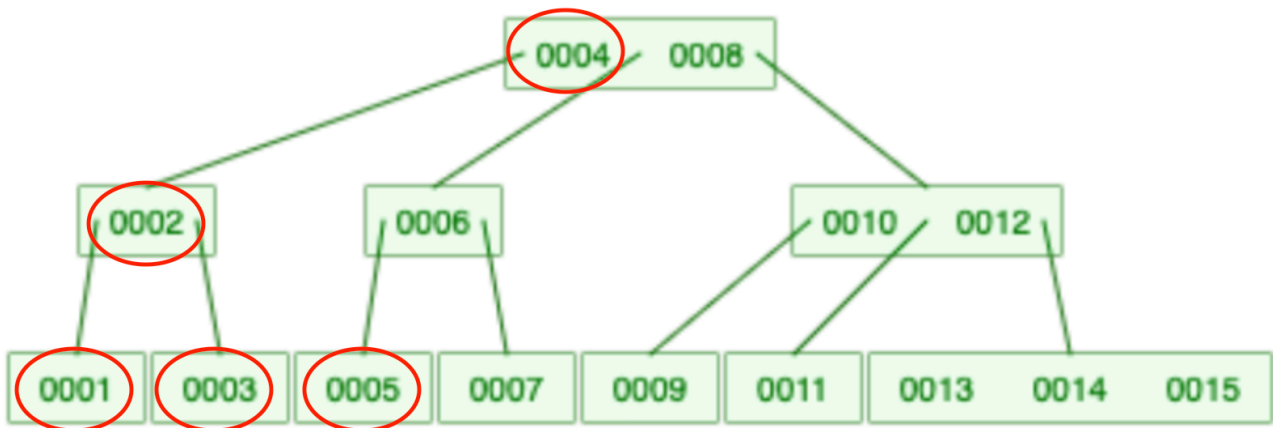
InnoDB uses a [B+Tree structure](#) for its indexes. - [Jeremy Cole](#)

He built multiple forks of MySQL, for example, Twitter MySQL, he was the head of Cloud SQL at Google and worked on the internals of MySQL and InnoDB.

Problems with B-Trees

There are two issues with a B-Tree. Imagine a query such as this one:

```
select *
from users
where id in (1,2,3,4,5)
```



It takes at least 8 steps to find these 5 values:

- From 4 to 2
- From 2 to 1
- From 1 back to 2
- From 2 to 3
- From 3 to 2
- From 2 to 4
- From 4 to 6
- From 6 to 5

So a B-Tree is not the best choice for range queries.

The other problem is wasting space. There's one thing I didn't mention so far. In this example, only the ID is present on the tree. Because this example is a primary key index. But of course, in real life, we add indexes to other columns such as usernames, created_at, other dates, and so on. These values are also stored in the tree.

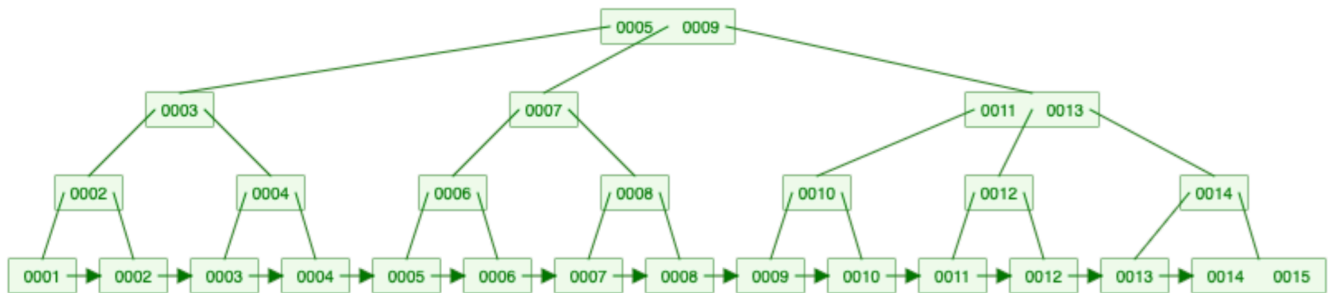
An index has the same number of elements as the table so the size of it can be huge if the table is big enough. This makes a B-Tree less optimal to load into memory.

B+ Trees

As the available size of the memory grew in servers, developers wanted to load the index into memory to achieve really good performance. B-Trees are amazing, but as we discussed they have two problems: size and range queries.

Surprisingly enough, one simple property of a B-Tree can lead us to a solution: most nodes are leaf nodes. The tree above contains 15 nodes and 9 of them are leaves. This is 60%.

Sometime around 1973, someone probably at IBM came up with the idea of a B+ Tree:



This tree contains the same numbers from 1 to 15. But it's considerably bigger than the previous B-Tree, right?

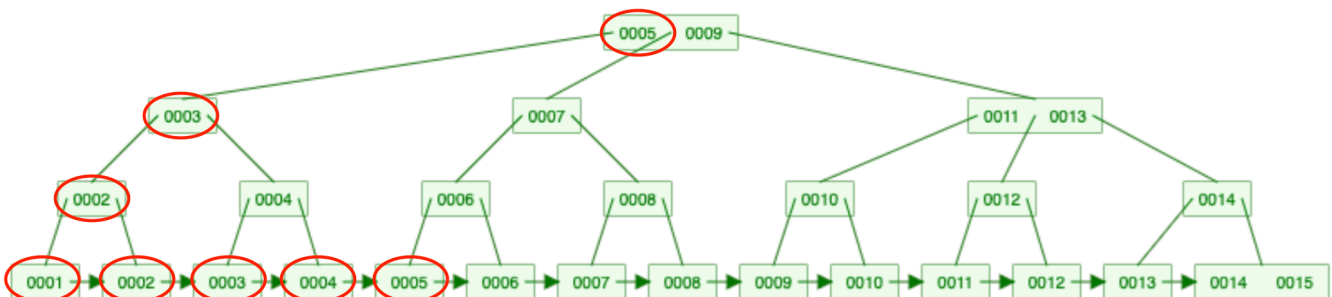
There are two important changes compared to a B-Tree:

- Every value is present as a leaf node. At the bottom of the tree, you can see every value from 1 to 15
- Some nodes are duplicated. For example, number 2 is present twice on the left side. Every node that is not a leaf node in a B-Tree is duplicated in a B+ Tree (since they are also inserted as leaf nodes)
- Leaf nodes form a linked list. This is why you can see arrows between them.
- Every non-leaf node is considered as a "routing" node.

With the linked list, the range query problem is solved. Given the same query:

```
select *
from users
where id in (1,2,3,4,5)
```

This is what the process looks like:



Once you have found the first leaf node, you can traverse the linked list since it's ordered. Now, in this specific example, the number of operations is the same as before, but in real life, we don't have a tree of 15 but instead 150,000 elements. In these cases, linked list traversal is way better.

So the range query problem is solved. But how does an even bigger tree help reduce the size?

The trick is that routing nodes **do not contain values**. They don't hold the usernames, the timestamps, etc. They are routing nodes. They only contain pointers so they are really small items. All the data is stored at the bottom level. **Only leaf nodes contain** our data.

Leaf nodes are not loaded into memory but only routing nodes. As weird as it sounds at first according to [PostgreSQL](#) this way the routing nodes take up only 1% of the overall size of the tree. Leaf nodes are the remaining 99%:

Each internal page (*comment: they are the routing nodes*) contains tuples (*comment: MySQL stores pointers to rows*) that point to the next level down in the tree. Typically, **over 99% of all pages are leaf pages**. - [PostgreSQL Docs](#)

So database engines typically only keep the routing nodes in memory. They can travel to find the necessary leaf nodes that contain the actual data. If the query doesn't need other columns it's essentially can be served using only the index. If the query needs other columns as well, MySQL reads it from the disk using the pointers in the leaf node.

I know this was a long introduction but in my opinion, this is the bare minimum we should know about indexes. Here are some closing thoughts:

- Both B-Trees and B+ trees have $O(\log n)$ time complexity for search, insert, and delete but as we've seen range queries perform better in a B+ tree.
- MySQL (and Postgres) uses B+ Trees, not B-Trees.
- The nodes in real indexes do not contain 3 or 4 keys as in these examples. They contain **thousands** of them. To be precise, a node matches the page size in your OS. This is a standard practice in databases. [Here you can see](#) in MySQL's source code documentation that the `btr_get_size` function, for example, returns the size of the index expressed as the number of pages. `btr_` stands for `btree`.
- Interestingly enough MongoDB uses B-Trees instead of B+ Trees as stated in the [documentation](#). Probably this is why Discord moved to Cassandra. They wrote this on their blog:

Around November 2015, we reached 100 million stored messages and at this time we started to see the expected issues appearing: the data and **the index could no longer fit in RAM** and latencies started to become unpredictable. It was time to migrate to a database more suited to the task. - [Discord Blog](#)

This chapter continues in the book with access types and practical examples.

Chunking large datasets

When it comes to working with larger datasets one of the best you can apply to any problem is chunking. Divide the dataset into smaller chunks and process them. It comes in many different forms. In this chapter, we're going review a few of them but the basic idea is always the same: divide your data into smaller chunks and process them.

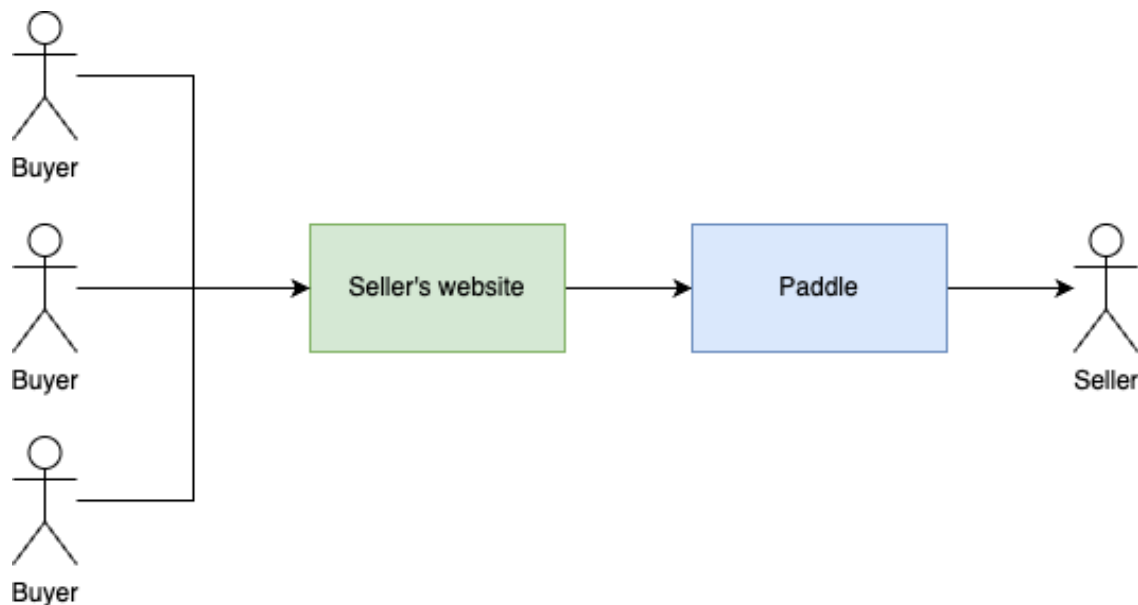
Chunking large datasets

When it comes to working with larger datasets one of the best you can apply to any problem is chunking. Divide the dataset into smaller chunks and process them. It comes in many different forms. In this chapter, we're going to review a few of them but the basic idea is always the same: divide your data into smaller chunks and process them.

Exports

Exporting to CSV or XLS and importing from them is a very common feature in modern applications.

I'm going to use a finance application as an example. Something like Paddle, Gumroad. They are a merchant of records that look like this:



This is what's happening:

- A seller (or content creator) uploads a product to Paddle
- They integrate Paddle into their landing page
- Buyers buy the product from the landing page using a Paddle checkout form
- Paddle pays the seller every month

I personally use Paddle to sell my books and SaaS and it's a great service. The main benefit is that you don't have to deal with hundreds or thousands of invoices and VAT ramifications. Paddle handles it for you. They send an invoice to the buyer and apply the right amount of VAT based on the buyer's location. They also handle VAT ramifications. You, as the seller, don't have to deal with any of that stuff. They just send you the money once every month and you have only one invoice. It also provides nice dashboards and reports.

Every month they send payouts to their users based on the transactions. They also send a CSV that contains all the transactions in the given month.

This is the problem we're going to imitate in this chapter. Exporting tens of thousands of transactions in an efficient way.

This is what the `transactions` table looks like:

id	product_id	quantity	revenue	balance_earnings	payout_id	stripe_id	user_id	created_at
1	1	1	3900	3120	NULL	acac83e2	1	2024-04-22 13:59:07
2	2	1	3900	3120	NULL	...	1	2024-04-17 17:43:12

These are two transactions for user #1. I shortened some UUIDs so the table fits the page better. Most columns are pretty easy to understand. Money values are stored in cent values so `3900` means `$39`. There are other rows as well, but they are not that important.

When it is payout time, a job queries all transactions in a given month for a user, creates a `Payout` object, and then sets the `payout_id` in this table. This way we know that the given transaction has been paid out. The same job exports the transactions for the user and sends them via e-mail.

`laravel-excel` is one of the most popular frameworks when it comes to imports/exports so we're going to use it in the first example.

This is what a typical export looks like:

```
namespace App\Exports;

class TransactionsSlowExport implements FromCollection, WithMapping, WithHeadings
{
    use Exportable;

    public function __construct(
        private User $user,
        private DateInterval $interval,
    ) {}

    public function collection()
    {
        return Transaction::query()
            ->where('user_id', $this->user->id)
            ->whereBetween('created_at', [
                $this->interval->startDate,
                $this->interval->endDate,
            ])
            ->get();
    }

    public function map($row): array
    {
        return [
            $row->uuid,
            Arr::get($row->product_data, 'title'),
            $row->quantity,
            MoneyForHuman::from($row->revenue)->value,
            MoneyForHuman::from($row->fee_amount)->value,
            MoneyForHuman::from($row->tax_amount)->value,
            MoneyForHuman::from($row->balance_earnings)->value,
            $row->customer_email,
            $row->created_at,
        ];
    }
}
```

```

}

public function headings(): array
{
    return [
        '#',
        'Product',
        'Quantity',
        'Total',
        'Fee',
        'Tax',
        'Balance earnings',
        'Customer e-mail',
        'Date',
    ];
}
}

```

I've seen dozens of exports like this one over the years. It creates a CSV from a collection. In the `collection` method, you can define your collection which is 99% of the time the result of a query. In this case, the collection contains `Transaction` models. Nice and simple.

However, an export such as this one, has two potential problems:

- The `collection` method runs a single query and loads **each and every** transaction into memory. The moment you exceed x number of models your process will die because of memory limitations. x of course varies highly.
- If your collection is not that big and the export made it through the query, the `map` function will run for **each and every** transaction. If you execute only one query here, it'll run `n` times where `n` is the number of rows in your CSV. This is the breeding ground for N+1 problems.

Be aware of these things because it's pretty easy to kill your server with a poor export.

The above job has failed with only 2,000 transactions:

```

select count(*)
from transactions
where user_id = 1
and created_at between "2024-04-10" and "2024-04-22"

```

line 4, column 53, location 104

count(*)
1958

1,958 to be precise. The result is `Allowed memory size exhausted`:

```

2024-04-22 19:41:34 Maatwebsite\Excel\Jobs\QueueExport 99 ..... RUNNING
PHP Fatal error: Allowed memory size of 134217728 bytes exhausted (tried to allocate 7961725 bytes) in /Users/joomartin/code/paddle-clone/vendor/
laravel/framework/src/Illuminate/Database/Connection.php on line 580

Symfony\Component\ErrorHandler\Error\FatalError

Allowed memory size of 134217728 bytes exhausted (tried to allocate 7961725 bytes)

at vendor/laravel/framework/src/Illuminate/Database/Connection.php:580
 576         $this->bindValues($statement, $this->prepareBindings($bindings));
 577
 578         $this->recordsHaveBeenModified();
 579
 580     return $statement->execute();
 581 }

```

As you can see, it is executed in a worker. This is possible by two things:

- The export uses the `Exportable` trait from the package, which has a `queue` function
- The method that runs the export uses this `queue` method:

```

new TransactionsExport(
    $user,
    $interval,
)
->queue($report->relativePath())
->chain([
    new NotifyUserAboutExportJob($user, $report),
]);

```

This is how you can make an export or import queueable.

Fortunately, there's a much better export type than `FromCollection`, it is called `FromQuery`. This export does not define a `Collection` but a DB query instead that will be executed in chunks by `laravel-excel`.

This is how we can rewrite the export class:

```

namespace App\Exports;

class TransactionsExport implements FromQuery, WithHeadings, WithCustomChunkSize,
WithMapping
{
    use Exportable;

    public function __construct(
        private User $user,
        private DateInterval $interval,
    ) {}

    public function query()
    {
        return Transaction::query()
            ->select([

```

```

        'uuid',
        'product_data',
        'quantity',
        'revenue',
        'fee_amount',
        'tax_amount',
        'balance_earnings',
        'customer_email',
        'created_at',
    ])
    ->where('user_id', $this->user->id)
    ->whereBetween('created_at', [
        $this->interval->startDate->date,
        $this->interval->endDate->date,
    ])
    ->orderBy('created_at');
}

public function chunkSize(): int
{
    return 250;
}

public function headings(): array
{
    // Same as before
}

public function map($row): array
{
    // Same as before
}
}

```

Instead of returning a `Collection` the `query` method returns a query builder. In addition, you can also use the `chunkSize` method. It works hand in hand with `Exportable` and `FromQuery`:

- Queued exports (using the `Exportable` trait and the `queue` method) are processed in chunks
- If the export implements `FromQuery` the number of jobs is calculated by `query()->count() / chunkSize()`

So in the `chunkSize` we can control how many jobs we want. For example, if we have 5,000 transactions for a given user and `chunkSize()` returns 250 which means that 20 jobs will be dispatched each processing 250 transactions. Unfortunately, I cannot give you exact numbers. It all depends on your specific use case. However, it's a nice way to fine-tune your export.

Using the techniques above, exporting 10k transactions is a walk in the park:

```
select count(*)
from transactions
where user_id = 1
and created_at between "2023-04-22" and "2024-04-22"

line 4, column 53, location 104
```

count(*)	
9847	

9,847 to be precise but the jobs are running smoothly. There are 40 jobs each processing 250 transactions:

2024-04-22 20:05:28	Maatwebsite\Excel\Jobs\AppendQueryToSheet 164	1s	DONE
2024-04-22 20:05:28	Maatwebsite\Excel\Jobs\AppendQueryToSheet 165		RUNNING
2024-04-22 20:05:29	Maatwebsite\Excel\Jobs\AppendQueryToSheet 166	1s	DONE
2024-04-22 20:05:29	Maatwebsite\Excel\Jobs\AppendQueryToSheet 167		RUNNING
2024-04-22 20:05:30	Maatwebsite\Excel\Jobs\AppendQueryToSheet 168	1s	DONE
2024-04-22 20:05:30	Maatwebsite\Excel\Jobs\AppendQueryToSheet 169		RUNNING
2024-04-22 20:05:31	Maatwebsite\Excel\Jobs\AppendQueryToSheet 170	1s	DONE
2024-04-22 20:05:31	Maatwebsite\Excel\Jobs\AppendQueryToSheet 171		RUNNING
2024-04-22 20:05:32	Maatwebsite\Excel\Jobs\AppendQueryToSheet 172	1s	DONE
2024-04-22 20:05:32	Maatwebsite\Excel\Jobs\AppendQueryToSheet 173		RUNNING
2024-04-22 20:05:33	Maatwebsite\Excel\Jobs\AppendQueryToSheet 174	1s	DONE
2024-04-22 20:05:33	Maatwebsite\Excel\Jobs\AppendQueryToSheet 175		RUNNING
2024-04-22 20:05:34	Maatwebsite\Excel\Jobs\CloseSheet 169	1s	DONE
2024-04-22 20:05:34	Maatwebsite\Excel\Jobs\StoreQueuedExport 170		RUNNING
2024-04-22 20:05:34	Maatwebsite\Excel\Jobs\StoreQueuedExport 171	23.15ms	DONE
2024-04-22 20:05:34	App\Jobs\NotifyUserAboutExportJob 171		RUNNING
2024-04-22 20:05:34	App\Jobs\NotifyUserAboutExportJob 171	13.46ms	DONE

The last jobs `laravel-excel` runs are `CloseSheet` and `StoreQueuedExport`.

This chapter continues in the book with imports, generators, LazyCollections, file operations, and DB delete operations.

Miscellaneous

fpm processes

php-fpm comes with a number of configurations that can affect the performance of our servers. These are the most important ones:

- `pm.max_children`: This directive sets the maximum number of fpm child processes that can be started. This is similar to `worker_processes` in nginx.
- `pm.start_servers`: This directive sets the number of fpm child processes that should be started when the fpm service is first started.
- `pm.min_spare_servers`: This directive sets the minimum number of idle fpm child processes that should be kept running to handle incoming requests.
- `pm.max_spare_servers`: This is the maximum number of idle fpm child processes.
- `pm.max_requests`: This directive sets the maximum number of requests that an fpm child process can handle before it is terminated and replaced with a new child process. This is similar to the `--max-jobs` option of the `queue:work` command.

So we can set `max_children` to the number of CPUs, right? Actually, nope.

The number of php-fpm processes is often calculated **based on memory** rather than CPU because PHP processes are typically memory-bound rather than CPU-bound.

When a PHP script is executed, it loads into memory and requires a certain amount of memory to run. The more PHP processes that are running simultaneously, the more memory will be consumed by the server. If too many PHP processes are started, the server may run out of memory and begin to swap, which can lead to performance issues.

TL;DR: if you don't have some obvious performance issue in your code php usually consumes more memory than CPU.

So we need a few pieces of information to figure out the correct number for the `max_children` config:

- How much memory does your server have?
- How much memory does a php-fpm process consume on average?
- How much memory does your server need just to stay alive?

Here's a command that will give you the average memory used by fpm processes:

```
ps -ylC php-fpm8.1 --sort:rss
```

- `ps` is a command used to display information about running processes.
- `-y` tells `ps` to display the process ID (PID) and the process's controlling terminal.
- `-l` instructs `ps` to display additional information about the process, including the process's state, the amount of CPU time it has used, and the command that started the process.
- `-C php-fpm8.1` tells `ps` to only display information about processes with the name `php-fpm8.1`.
- `--sort:rss`: will sort the output based on the amount of resident set size (RSS) used by each process.

What the hell is the resident set size? It's a memory utilization metric that refers to the amount of physical memory currently being used by a process. It includes the amount of memory that is allocated to the process and **cannot be shared** with other processes. This includes the process's executable code, data, and stack space, as well as any memory-mapped files or shared libraries that the process is using.

It's called "resident" for a reason. It shows the amount of memory that cannot be used by other processes. For example, when you run `memory_get_peak_usage()` in PHP it only returns the memory used by the PHP script. On the other hand, RSS measures the total memory usage of the entire process.

The command will spam your terminal with an output such as this:

```
root@askmycontent:~# ps -ylC php-fpm8.1 --sort:rss
```

S	UID	PID	PPID	C	PRI	NI	RSS	SZ	WCHAN	TTY	TIME	CMD
S	0	13789	1	0	80	0	25088	62620	ep_poll ?		00:00:18	php-fpm8.1
S	33	18313	13789	0	80	0	42440	81729	skb_wa ?		00:00:08	php-fpm8.1
S	33	18317	13789	0	80	0	43156	81750	skb_wa ?		00:00:13	php-fpm8.1
S	33	13791	13789	0	80	0	43984	81745	skb_wa ?		00:00:13	php-fpm8.1

```
root@askmycontent:~#
```

The RSS column shows the memory usage. From 25Mb 43MB in this case. The first line (which has significantly lower memory usage) is usually the master process. We can take that out of the equation and say the average memory used by a php-fpm worker process is 43MB.

However, here are some numbers from a production (older) app:

```
S 1000 13848 1 0 80 0 74408 137609 inet_c ? 00:00:05 php-fpm7.3
S 1000 13841 1 0 80 0 74460 137601 inet_c ? 00:00:06 php-fpm7.3
S 1000 13838 1 0 80 0 74768 137729 inet_c ? 00:00:08 php-fpm7.3
S 1000 13862 1 0 80 0 75000 139088 inet_c ? 00:00:01 php-fpm7.3
S 1000 13846 1 0 80 0 75640 137616 inet_c ? 00:00:04 php-fpm7.3
S 1000 13833 1 0 80 0 76012 137722 inet_c ? 00:00:10 php-fpm7.3
S 1000 13837 1 0 80 0 76916 137746 inet_c ? 00:00:10 php-fpm7.3
S 1000 13806 1 0 80 0 77484 137746 inet_c ? 00:00:16 php-fpm7.3
S 1000 13827 1 0 80 0 80072 139535 inet_c ? 00:00:12 php-fpm7.3
S 1000 13839 1 0 80 0 81156 157003 inet_c ? 00:00:07 php-fpm7.3
S 1000 13840 1 0 80 0 81652 156791 inet_c ? 00:00:07 php-fpm7.3
S 1000 13849 1 0 80 0 81900 156991 inet_c ? 00:00:06 php-fpm7.3
S 1000 13836 1 0 80 0 85164 158663 inet_c ? 00:00:08 php-fpm7.3
```

Yes, these are 130MB+ numbers.

The next question is how much memory does your server need just to stay alive? This can be determined using `htop`:



As you can see from the load average, right now nothing is happening on this server but it uses ~700MB of RAM. This memory is used by Linux, PHP, MySQL, Redis, and all the system components installed on the machine.

So the answers are:

- This server has 2GB of RAM
- It needs 700MB to survive
- On average an fpm process uses 43MB of RAM

This means there is 1.3GB of RAM left to use. So we can spin up $1300/30=30$ fpm processes.

It's a good practice to decrease the available RAM by at least 10% as a kind of "safety margin". So let's calculate with 1.17GB of RAM: $1170/37=28$.

So on this particular server, I can probably run 25-30 fpm processes.

Here's how we can determine the other values:

Config	General	This example
pm.max_children	As shown above	28
pm.start_servers	~25% of max_children	7
pm.min_spare_servers	~25% of max_children	7
pm.max_spare_servers	~75% of max_children	21

To be completely honest, I'm not sure how these values are calculated but they are the "standard" settings. You can search these configs on the web and you probably run into an article suggesting similar numbers. By the way, there's also a calculator [here](#).

To configure these values you need to edit the `fpm` config file which in my case is located in `/etc/php/8.1/fpm/pool.d/www.conf`:

```
pm.max_children = 28
pm.start_servers = 7
pm.min_spare_servers = 7
pm.max_spare_servers = 21
```

After that, you need to restart `fpm`:


```
systemctl restart php8.1-fpm.service
```

Changing the number of children processes requires a full `restart` since fpm needs to kill and spawn processes.

This chapter continues in the book with nginx caches, MySQL slow query logs, Docker resource limits, etc